

ĐẠI HỌC BÁCH KHOA HÀ NỘI



ĐỒ ÁN TỐT NGHIỆP

THIẾT KẾ THIẾT BỊ ĐỌC GHI MÀN HÌNH CỘT BƠM XĂNG DẦU

NGUYỄN THÁI SƠN

son.nt212951@sis.hust.edu.vn

Ngành Kỹ thuật Điều khiển và Tự động hóa

Giảng viên hướng dẫn: PGS.TS. Nguyễn Quốc Cường

Chữ ký của GVHD

Khoa: Tự động hóa

Trường: Điện - Điện tử

Hà Nội, 3/2025

**NHIỆM VỤ
ĐỒ ÁN TỐT NGHIỆP**

Họ và tên sinh viên: Nguyễn Thái Sơn

Khóa: K66

Trường: Điện – Điện tử

Ngành: KT ĐK & TĐH

1. Tên đề tài

Thiết kế thiết bị đọc ghi màn hình cột bơm xăng dầu

2. Nội dung đề tài

Thiết kế và triển khai hệ thống gồm phần cứng và phần mềm để thu dữ liệu hiển thị trên màn hình cây xăng, giải mã và lưu các lượt bơm xăng. Lấy mẫu thông tin màn hình với tần số 100Hz. Có kết nối mạng LAN với máy tính nội bộ. Thiết bị có giao diện giám sát và điều khiển dạng Desktop App.

Các công việc bao gồm:

- Thiết kế mạch thu dữ liệu màn hình.
- Thu thập và phân tích tín hiệu gửi tới màn hình.
- Thiết kế và triển khai firmware cho thiết bị thu dữ liệu màn hình. Cụ thể là lập trình cho vi điều khiển STM32 tại khối xử lý trung tâm.
- Thiết kế và triển khai phần mềm điều khiển và giải mã. Cụ thể là viết phần mềm trên máy tính thực hiện giải mã dữ liệu màn hình, phát hiện phiên bơm và điều khiển relay cho thiết bị.
- Triển khai cơ sở dữ liệu và giao diện hiển thị dữ liệu màn hình.
- Chạy kiểm thử hệ thống tại hiện trường.

3. Thời gian giao đề tài: 18/02/2025

4. Thời gian hoàn thành: 18/6/2025

Ngày 18 tháng 6 năm 2025

CÁN BỘ HƯỚNG DẪN

Nguyễn Quốc Cường

LỜI CẢM ƠN

Đây là mục tùy chọn, nên viết phần cảm ơn ngắn gọn, tránh dùng các từ sáo rỗng.

Hà Nội, ngày 24 tháng 03 năm 2025

Sinh viên thực hiện

Nguyễn Thái Sơn

TÓM TẮT ĐỒ ÁN

Đồ án này đưa ra phương án thiết kế và triển khai một thiết bị đọc ghi màn hình cột bơm xăng dầu, phục vụ cho các cột bơm xăng dầu cũ không có chức năng xuất hóa đơn trên thị trường.

Chức năng chính của thiết bị bao gồm:

- Đọc và giải mã màn hình cây xăng theo thời gian thực
- Phát hiện các phiên bơm hoàn chỉnh và lưu vào cơ sở dữ liệu. Làm tiền đề cho việc xuất hóa đơn điện tử sau này.

Thiết bị giải mã cụ thể một loại cột bơm là cột hãng ZCheng với dữ liệu màn hình theo giao thức SPI. Thiết bị được lắp đặt trực tiếp vào cột bơm, có phần mềm giải mã, cơ sở dữ liệu và giao diện người dùng đi kèm.

Các công cụ chính được sử dụng bao gồm:

- Mạch cứng: STM32f407vet6, Chip W5500, nguồn POE Switch, các bộ đệm tín hiệu và cách ly quang
- Phần mềm giải mã: Ngôn ngữ C++, Win32 API, thư viện Boost (C++)
- Lưu trữ: SQLite database.
- Giao diện người dùng: Nền tảng NodeJS, ngôn ngữ TypeScript, thư viện sử dụng bao gồm Electron, ReactJS.

Kết quả thực nghiệm đã cho thấy hệ thống chạy ổn định trong thời gian dài và đáp ứng đủ các chức năng cơ bản đã đề ra. Hệ thống cũng đưa ra kiến trúc mở để dễ dàng mở rộng, thêm chức năng và thuận tiện cho giải mã các loại màn hình khác về sau.

MỤC LỤC

DANH MỤC KÝ HIỆU VÀ CHỮ VIẾT TẮT	i
DANH MỤC HÌNH VẼ	iii
DANH MỤC BẢNG BIỂU	iv
CHƯƠNG 1. GIỚI THIỆU CHUNG	1
1.1 Vai trò của thiết bị đọc ghi màn hình cột bơm xăng dầu	1
1.2 Khái quát về yêu cầu và chức năng của hệ thống	1
1.3 Phạm vi nghiên cứu	2
1.4 Phương pháp nghiên cứu	3
1.5 Cấu trúc đồ án	4
1.6 Kết luận chương	4
CHƯƠNG 2. TỔNG QUAN LÝ THUYẾT VÀ CÔNG NGHỆ	5
2.1 Tổng quan về chủ đề nghiên cứu	5
2.1.1 Lịch sử phát triển cột bơm xăng dầu [1]	5
2.1.2 Màn hình cột bơm xăng dầu	6
2.2 Cơ sở lý thuyết	7
2.2.1 LED 7 thanh	7
2.2.2 Giao thức SPI	9
2.2.3 Mạng LAN	10
2.3 Kiến trúc giao thức TCP/IP	11
2.3.1 Giao thức TCP	13
2.3.2 Giao thức UDP	15
2.4 Các công nghệ liên quan	16
2.4.1 Vi điều khiển STM32	16
2.4.2 Ngôn ngữ lập trình C/C++	17
2.4.3 Môi trường phát triển phần mềm NodeJS	18
2.5 Lựa chọn công nghệ phù hợp	19
2.5.1 Chip Ethernet để kết nối với mạng LAN	19
2.5.2 Vi điều khiển trung tâm cho thiết bị đọc màn hình	21
2.6 Kết luận chương	22

CHƯƠNG 3. THIẾT KẾ HỆ THỐNG	23
3.1 Phân tích yêu cầu hệ thống	23
3.1.1 Thiết bị đọc ghi màn hình	23
3.1.2 Phần mềm giải mã	23
3.1.3 Hệ thống mạng	24
3.2 Mô hình thiết kế tổng thể	24
3.2.1 Sơ đồ khối chức năng thiết bị đọc ghi màn hình (Device) . . .	24
3.2.2 Sơ đồ khối chức năng phần mềm giải mã (Device Service) . .	25
3.3 Thiết kế phần cứng thiết bị đọc màn hình	26
3.3.1 Sơ đồ khối phần cứng thiết bị đọc màn hình	26
3.3.2 Lựa chọn linh kiện và vẽ sơ đồ nguyên lý	26
3.4 Thiết kế firmware thiết bị đọc màn hình	30
3.4.1 Kiến trúc triển khai firmware	30
3.4.2 Trình tự giao tiếp giữa các khối	31
3.5 Thiết kế phần mềm giải mã Device Service	33
3.5.1 Kiến trúc triển khai phần mềm	33
3.5.2 Trình tự giao tiếp và thuật toán cho các khối	34
3.6 Giao diện điều khiển cho người dùng	38
3.7 Kết luận chương	40
CHƯƠNG 4. TRIỂN KHAI VÀ THỬ NGHIỆM	41
4.1 Quy trình triển khai	41
4.1.1 Mạch phần cứng thiết bị đọc màn hình	41
4.1.2 Firmware thiết bị đọc màn hình	42
4.1.3 Phần mềm giải mã Device Service	48
4.1.4 Viết phần mềm Client UI hiển thị các phiên bơm	56
4.2 Thử nghiệm và đánh giá kết quả thực tế	59
4.3 Kết luận chương	62
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	63
5.1 Tóm tắt kết quả đã đạt được	63
5.2 Những hạn chế còn tồn đọng	63
5.3 Định hướng phát triển trong tương lai	63
TÀI LIỆU THAM KHẢO	64

DANH MỤC KÝ HIỆU VÀ CHỮ VIẾT TẮT

HESS	Hybrid Energy Storage System
SC	Super Capacitor
EMS	Energy Management Strategy

DANH MỤC HÌNH VẼ

Hình 1.1.	Minh họa mục tiêu đầu ra của hệ thống	1
Hình 1.2.	Màn hình và mạch phần cứng màn hình cây xăng hãng ZCheng	4
Hình 2.1.	Nguyên lý hoạt động các cột bơm xăng dầu phổ biến	6
Hình 2.2.	Màn hình hiển thị cột bơm ZCheng sử dụng các LED 7 thanh	7
Hình 2.3.	Cấu trúc của LED 7 thanh	8
Hình 2.4.	Tổ chức mạng LAN	10
Hình 2.5.	Kiến trúc 4 tầng của mô hình TCP/IP	12
Hình 2.6.	Quy trình bắt tay ba bước trong TCP	14
Hình 2.7.	Cấu trúc gói tin TCP	14
Hình 2.8.	Cấu trúc gói tin UDP	15
Hình 2.9.	KIT phát triển STM32F407VET6 ARM Cortex-M4	17
Hình 2.10.	Module Ethernet W5500	21
Hình 3.1.	Các thành phần của hệ thống	23
Hình 3.2.	Sơ đồ khối chức năng thiết bị đọc ghi màn hình (Device)	24
Hình 3.3.	Sơ đồ khối chức năng phần mềm giải mã và chốt phiên bơm (Device Service)	25
Hình 3.4.	Sơ đồ khối triển khai phần cứng thiết bị đọc ghi màn hình	26
Hình 3.5.	Sơ đồ nguyên lý khối xử lý trung tâm	27
Hình 3.6.	Sơ đồ nguyên lý khối đầu vào tín hiệu	28
Hình 3.7.	Sơ đồ nguyên lý khối nguồn và giao tiếp Ethernet	29
Hình 3.8.	Sơ đồ khối triển khai firmware cho thiết bị	30
Hình 3.9.	Trình tự giao tiếp: Thiết lập kết nối tới phần mềm giải mã Device Service	31
Hình 3.10.	Trình tự giao tiếp: thu thập và đẩy dữ liệu màn hình	32
Hình 3.11.	Trình tự giao tiếp: cập nhật firmware từ xa (OTA)	34
Hình 3.12.	Sơ đồ khối triển khai phần mềm Device Service	35
Hình 3.13.	Trình tự giao tiếp cho phần mềm giải mã: Giải mã màn hình	36
Hình 3.14.	Trình tự giao tiếp phần mềm trong trường hợp điều khiển relay	37
Hình 3.15.	Trình tự giao tiếp phần mềm trong trường hợp thực hiện OTA	38
Hình 3.16.	Sơ đồ khối triển khai phần mềm giao diện	39
Hình 3.17.	Giao tiếp giữa các process của ứng dụng thông qua IPC	40
Hình 4.1.	Mạch PCB - Mặt trên	41
Hình 4.2.	Mạch PCB - Mặt ngang và mặt dưới	41

Hình 4.3.	Sản phẩm thiết bị đọc ghi màn hình cột bơm: mặt trên	42
Hình 4.4.	Sản phẩm thiết bị đọc ghi màn hình cột bơm: mặt ngang và dưới	42
Hình 4.5.	Triển khai: cấu hình các chân trong STM32CubeIDE	43
Hình 4.6.	Thuật toán xử lý ngắt nhận dữ liệu màn hình	44
Hình 4.7.	Thuật toán xử lý ngắt nhận dữ liệu yêu cầu Device Service	45
Hình 4.8.	Thuật toán cho chương trình chính	46
Hình 4.9.	Tạo C++ Win32 Project trên Visual Studio	48
Hình 4.10.	Thuật toán giải mã màn hình và phát hiện lượt bơm	50
Hình 4.11.	Các trường dữ liệu trong một Log hoặc Sublog	51
Hình 4.12.	Sơ đồ trạng thái cho cột bơm	52
Hình 4.13.	Thuật toán thực hiện OTA trên Device Service	54
Hình 4.14.	Device Service: Các file logs	56
Hình 4.15.	Mẫu dự án sử dụng Electron + Vite + ReactJS	57
Hình 4.16.	Các trường lưu trong bảng Log của Database	58
Hình 4.17.	Giao diện hiển thị Client UI	59
Hình 4.18.	Lắp thiết bị đọc màn hình vào cột bơm	60
Hình 4.19.	Cài đặt và cấu hình cho Device Service tại hiện trường	61

DANH MỤC BẢNG BIỂU

Bảng 2.1. Bảng mã nhị phân điều khiển LED 7 thanh hiển thị chữ số (Common Cathode)	8
Bảng 2.2. So sánh mô hình OSI và TCP/IP	13
Bảng 2.3. So sánh các chip Ethernet phổ biến	19
Bảng 4.1. Cấu trúc gói tin giữa Device và Device Service	47
Bảng 4.2. Danh sách các loại CMD trong enum Cmd_t	48
Bảng 4.3. Các loại sự kiện màn hình	51
Bảng 4.4. Mô tả các trạng thái máy (Machine State)	52
Bảng 4.5. Callback được gọi sau khi xác định trạng thái mới	53
Bảng 4.6. Danh sách các message_type của Device Service	55
Bảng 4.7. Bảng kết quả đánh giá kiểm thử hệ thống	61

CHƯƠNG 1. GIỚI THIỆU CHUNG

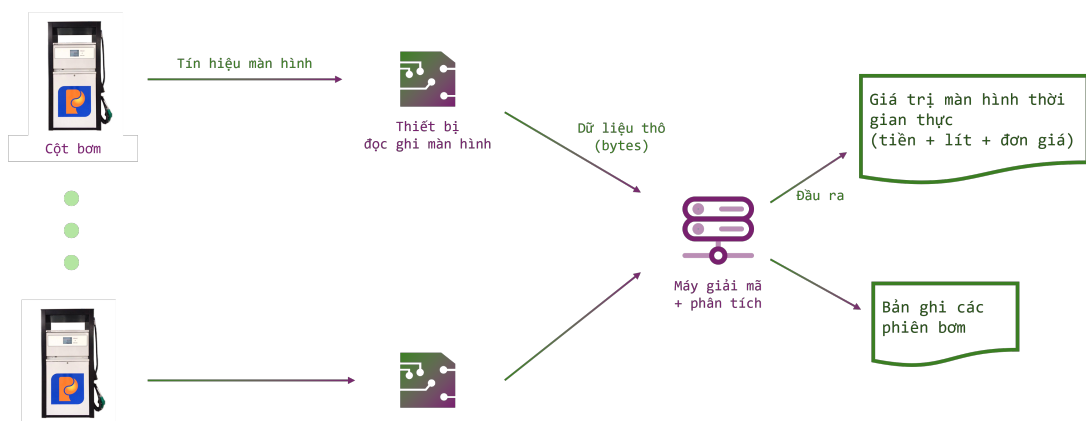
1.1 Vai trò của thiết bị đọc ghi màn hình cột bơm xăng dầu

Công nghệ số mang đến ngày càng nhiều lợi ích to lớn về mặt tiện dụng, hiệu quả và tiết kiệm thời gian cho doanh nghiệp cũng như người dùng. Từ đó, nhu cầu lớn về chuyển đổi số được phát sinh. Các thông tin giao dịch, hóa đơn trên giấy tờ ngày càng được thay thế bằng các giao dịch, hóa đơn điện tử, thuận tiện cho truyền tải thông tin, lưu trữ và quản lý. Các hệ thống trạm xăng và cột bơm cũng không phải ngoại lệ với nhu cầu đó, các giao dịch, lượt bơm xăng cũng cần được ghi lại dưới dạng dữ liệu số để gửi lên máy chủ của doanh nghiệp, thuận tiện lưu trữ và thống kê một cách chính xác, ổn định và giảm hoặc triệt tiêu những lỗi con người có thể gây ra.

Những cột bơm mới được bán gần đây cũng đã được sản xuất tích hợp chức năng xuất hóa đơn điện tử. Nhưng còn một lượng rất lớn những cây xăng với công nghệ cũ vẫn còn đạt tiêu chuẩn và sử dụng tốt được ở nhiều trạm bơm trên khắp cả nước, việc bỏ hẳn những cây xăng này đi thay bằng những cây xăng công nghệ mới sẽ tốn nhiều thời gian và chi phí đầu tư, gây lãng phí cũng như thải nhiều rác thải điện tử hơn cho môi trường. Từ đó xuất hiện nhu cầu cải tiến những cây xăng cũ để xuất được hóa đơn điện tử, chi phí và thời gian doanh nghiệp bỏ ra sẽ được tối ưu hơn.

Từ yêu cầu trực tiếp của doanh nghiệp, báo cáo này được viết để đưa ra phương án thiết kế và triển khai hệ thống có thể đọc ghi và giải mã, phân tích dữ liệu màn hình cây xăng, từ đó phát hiện được các lần giao dịch và lưu trữ, quản lý các lượt giao dịch - lượt bơm xăng đó.

1.2 Khái quát về yêu cầu và chức năng của hệ thống



Hình 1.1. Minh họa mục tiêu đầu ra của hệ thống

Mục tiêu là thiết kế thiết bị đọc ghi và giải mã màn hình cây xăng đáp ứng những nhu cầu sau:

1. Giám sát số liệu hiển thị trên cột bơm theo thời gian thực

Hệ thống có thể đọc và hiển thị trực tiếp dữ liệu cột bơm theo thời gian thực bao gồm: số tiền, số lít và đơn giá. Người dùng có thể theo dõi trạng thái hiển thị của cột bơm ngay tại phòng điều khiển mà không cần quan sát trực tiếp màn hình cột.

2. Phát hiện các lượt bơm hoàn chỉnh

Thông qua các số liệu thời gian thực (số tiền, số lít, đơn giá) của cột bơm, hệ thống phát hiện được đầu là bắt đầu và kết thúc của một lượt bơm xăng, thông báo một lượt bơm/hóa đơn hoàn chỉnh.

3. Lưu trữ và hiển thị các lượt bơm

Các lượt bơm được lưu trữ trong cơ sở dữ liệu, tạo cơ sở để có thể xuất các hóa đơn điện tử.

4. Hỗ trợ điều khiển từ xa

Việc ghi dữ liệu cột bơm có thể được điều khiển bật tắt từ xa, thông qua giao diện phần mềm.

5. Cập nhật phần mềm từ xa (OTA)

Hỗ trợ cập nhật phần mềm từ xa, cho phép người dùng cập nhật phiên bản mới từ giao diện người dùng để có tối ưu những tính năng, tiện ích mới nhất từ phiên bản phần cứng đã có.

Thông qua việc triển khai mục tiêu trên, đề tài hướng tới tạo ra một hệ thống đọc và giải mã màn hình cây xăng hoàn chỉnh, đáp ứng nhu cầu quan sát từ xa các cột bơm theo thời gian thực, tự động phát hiện và ghi lại được các lượt bơm, tạo tiền đề để phát triển các ứng dụng như là xuất hóa đơn điện tử, thanh toán không chạm, và các ứng dụng khác.

1.3 Phạm vi nghiên cứu

Thiết bị đọc ghi màn hình cột bơm xăng dầu có thể đọc ghi và giải mã loại cột bơm truyền thống có đặc điểm sau:

- Hãng sản xuất: ZCheng
- Loại cáp màn hình: 8P 3.96mm
- Số lượng chân cáp: 8
- Số chân tín hiệu của cáp: 3
- Giao thức: SPI
- Độ lớn frame: 22 byte
- Bit rate: 400Kb/s
- Tần số frame: 100Hz

Đây là loại cột bơm cũ phổ biến tại các trạm xăng trên thị trường. Phạm vi chức năng của hệ thống bao gồm:

1. Giám sát số liệu hiển thị trên cột bơm theo thời gian thực

Thu và giải mã dữ liệu SPI gửi tới màn hình thiết bị. Hiển thị giá trị màn hình (số tiền, số lít, đơn giá) theo thời gian thực trên giao diện người dùng.

2. Phát hiện các phiên bơm hoàn chỉnh

Sau khi bơm xăng, người vận hành dừng bơm một thời gian (5s) hoặc nhấn nút reset để bắt đầu phiên bơm mới, thiết bị phát hiện và ghi lại giá trị màn hình hiện tại thành một phiên bơm.

3. Lưu trữ và hiển thị các lượt bơm

Các bản ghi phiên bơm được lưu vào cơ sở dữ liệu ngay trên máy tính nội bộ và hiển thị lên giao diện quản lý

4. Hỗ trợ điều khiển từ xa

Thiết bị đọc màn hình cây xăng có 2 relay đóng ngắt có thể nối nối tiếp vào cột bơm. Từ giao diện quản lý, có thể điều khiển relay đóng ngắt bằng nút ấn trên giao diện.

5. Cập nhật phần mềm từ xa (OTA)

Từ giao diện quản lý, có thể tải (upload) file phần mềm dạng mã nhị phân, gửi cho thiết bị đọc màn hình để thiết bị tự cập nhật.

1.4 Phương pháp nghiên cứu

Bối cảnh màn hình cây xăng là các màn hình cũ, không có tài liệu cụ thể về thiết phần cứng và giao thức.

Do đó, cần khảo sát tín hiệu màn hình. Sử dụng mạch thu và phần mềm của Logic Analyzer để thu tín hiệu theo các bước sau:

- Sử dụng đồng hồ đo áp, xác định các chân nguồn, đất, tín hiệu và các chân có điện áp cao (để tránh cắm chân Logic Analyzer vào các chân có điện áp cao này).

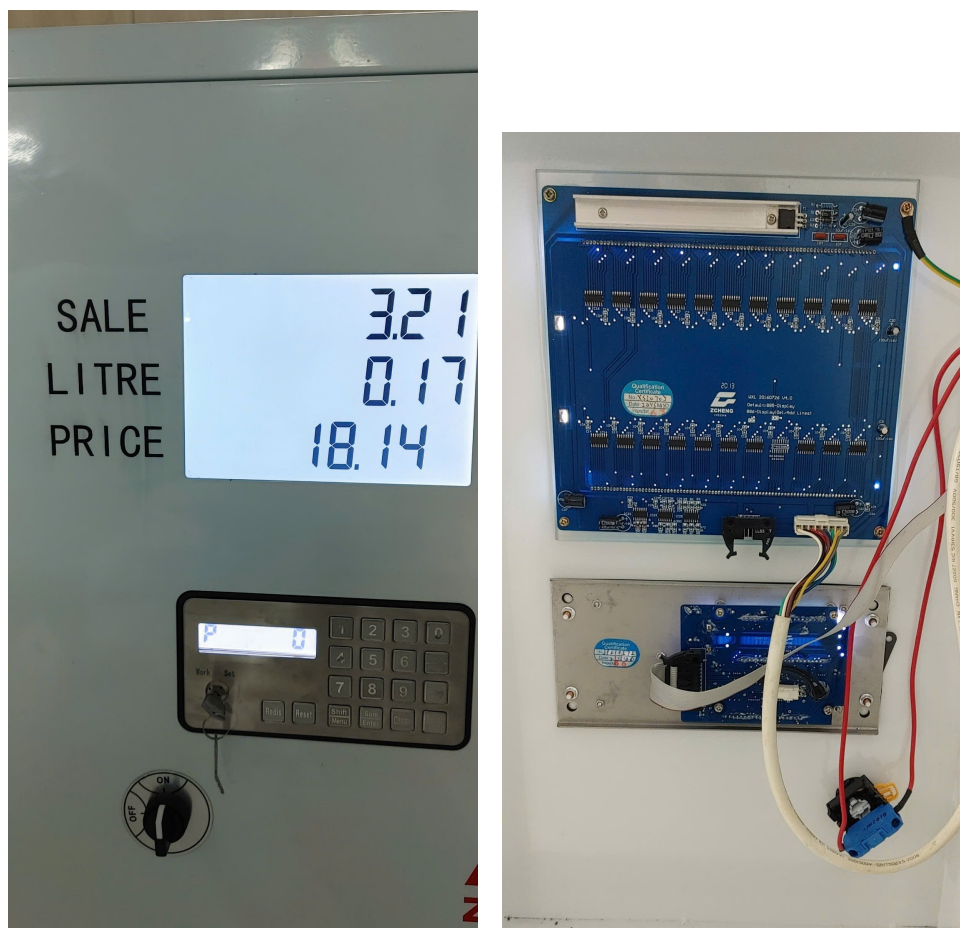
- Sử dụng mạch cứng và phần mềm Logic Analyzer nối vào các chân tín hiệu tới màn hình để thu các tín hiệu. Đồng thời, sử dụng máy quay quay lại màn hình hiển thị trong quá trình thu.

- Thực hiện các thao tác phổ biến: reset màn hình, bơm xăng, preset màn hình (đặt trước giá trị tiền hoặc lít cần bơm) rồi ghi lại tín hiệu và video màn hình trong quá trình vận hành thao tác.

- Khảo sát tín hiệu, so sánh tín hiệu với giá trị hiển thị trên màn hình để xác định:

- Vị trí các chân (nguồn, đất, cấp xung, dữ liệu, ...)
- Giao thức sử dụng (SPI), dấu hiệu bắt đầu và kết thúc một gói tin
- Ý nghĩa các byte trong gói tin

Từ đó, chế tạo mạch thu gói tin theo giao thức SPI và thiết kế phần mềm giải mã.



Hình 1.2. Màn hình và mạch phần cứng màn hình cây xăng hãng ZCheng

1.5 Cấu trúc đồ án

Nội dung được trình bày ở các chương tiếp theo bao gồm:

- Tổng quan lý thuyết và công nghệ: Trình bày các nền tảng lý thuyết và mô tả các công nghệ được sử dụng trong thiết kế.
- Thiết kế hệ thống sản phẩm: Bao gồm sơ đồ các khối chức năng của thiết bị đọc ghi màn hình và phần mềm giải mã, cùng với thiết kế chi tiết (mạch cứng và phần mềm) từng khối, trình tự giao tiếp giữa các khối.
- Triển khai và thử nghiệm: Thử nghiệm thiết bị trên màn hình cột bơm tại hiện trường, đánh giá kết quả.
- Kết luận và hướng phát triển.

1.6 Kết luận chương

Trong chương này, em đã đưa ra bối cảnh thực tế của bài toán, giải thích nhu cầu thực tế của thiết bị đọc ghi màn hình cây xăng, cùng với lợi ích mà nó mang lại trong quá trình phát triển và chuyển đổi số.

Đồng thời, chương này liệt kê các yêu cầu kỹ thuật cụ thể, cách tiếp cận bài toán từ đó đưa ra hướng thiết kế và triển khai tiếp theo.

CHƯƠNG 2. TỔNG QUAN LÝ THUYẾT VÀ CÔNG NGHỆ

2.1 Tổng quan về chủ đề nghiên cứu

2.1.1 Lịch sử phát triển cột bơm xăng dầu [1]

Cột bơm xăng dầu (fuel dispenser hay gas pump) là thiết bị thiết yếu trong hệ thống trạm nhiên liệu, được sử dụng để bơm nhiên liệu như xăng hoặc dầu diesel vào phương tiện giao thông. Trong suốt hơn một thế kỷ qua, thiết bị này đã trải qua nhiều giai đoạn cải tiến về mặt cơ khí, điện tử và an toàn.

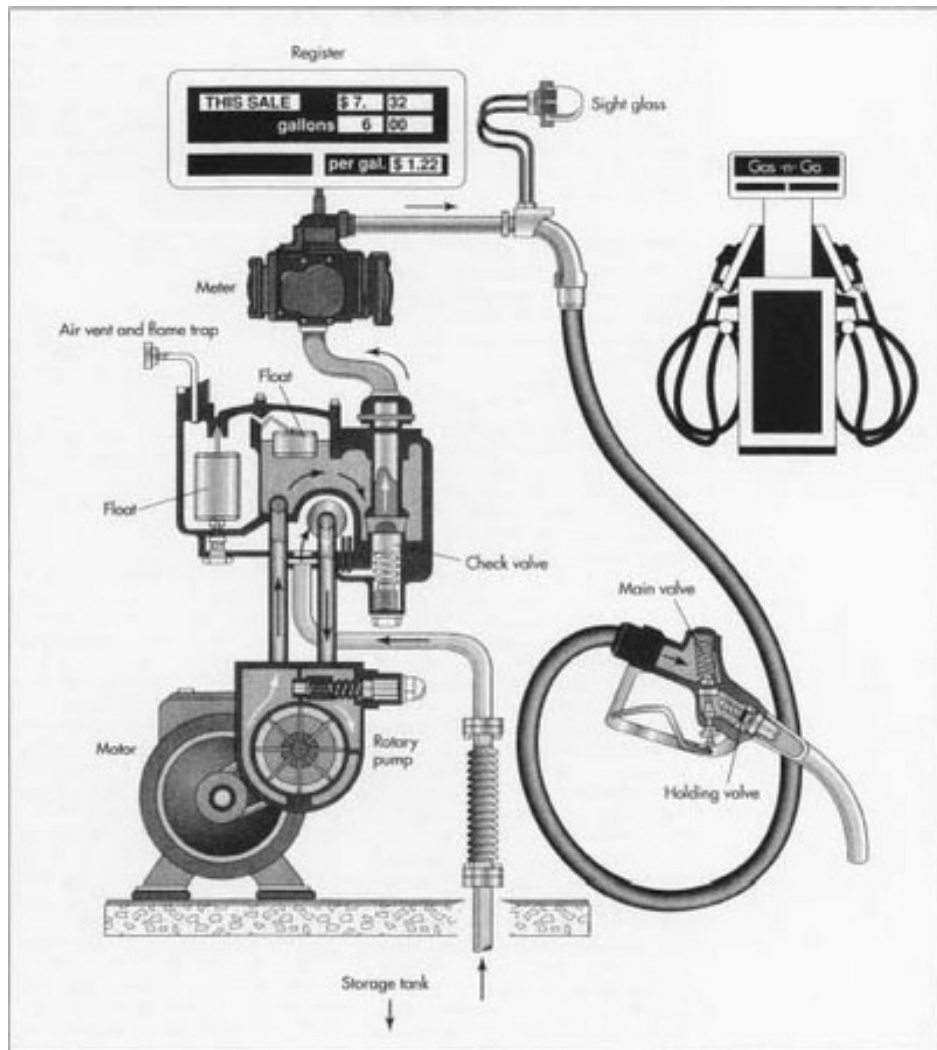
Cột bơm đầu tiên được phát minh vào năm 1885 bởi Sylvanus Bowser tại Fort Wayne, Indiana (Hoa Kỳ), ban đầu được thiết kế để bơm kerosene cho mục đích thắp sáng và nấu ăn. Sau đó, khi ô tô trở nên phổ biến vào đầu thế kỷ 20, thiết kế này nhanh chóng được cải tiến để phục vụ việc phân phối xăng.

Đến năm 1905, công ty Bowser đã giới thiệu dòng sản phẩm *Self-Measuring Gasoline Storage Pump* - thiết bị đầu tiên kết hợp bộ đo thể tích nhiên liệu, giúp người dùng biết được chính xác lượng nhiên liệu được bơm. Các phiên bản ban đầu có cấu trúc thủ công, sử dụng tay quay để bơm nhiên liệu, và có bình thủy tinh trong suốt phía trên nhằm đo thể tích bằng mắt thường.

Vào những năm 1920-1930, các công ty như Tokheim, Wayne và Gilbarco đã bắt đầu phát triển các trạm bơm hiện đại hơn, với bơm điện thay thế hệ thống tay quay. Bên cạnh đó, thiết kế cột bơm cũng bắt đầu được cải tiến để đảm bảo an toàn, chẳng hạn như sử dụng van ngắt tự động và ống bơm linh hoạt hơn nhằm tránh cháy nổ.

Sau Thế chiến II, đặc biệt từ những năm 1950 trở đi, cột bơm chuyển dần sang dạng thiết bị tự động hoá, có tích hợp đồng hồ đo cơ học, công tơ mét và bảng hiển thị giá thành. Tới những năm 1980, công nghệ vi xử lý bắt đầu được tích hợp, cho phép cột bơm giao tiếp với hệ thống thanh toán, quản lý tồn kho và giám sát từ xa.

Ngày nay, các cột bơm xăng dầu hiện đại không chỉ đảm bảo độ chính xác trong việc đo lường và phân phối nhiên liệu, mà còn được tích hợp các tính năng kỹ thuật số như: màn hình cảm ứng, thanh toán không tiếp xúc (contactless), nhận diện phương tiện tự động, kết nối internet để cập nhật giá xăng theo thời gian thực và thậm chí cung cấp quảng cáo trực tiếp trên màn hình bơm.



Hình 2.1. Nguyên lý hoạt động các cột bơm xăng dầu phổ biến

Sự phát triển của cột bơm xăng dầu phản ánh sự tiến bộ chung của ngành giao thông và công nghệ nhiên liệu, với mục tiêu cuối cùng là tối ưu hóa hiệu quả, an toàn và trải nghiệm người dùng.

2.1.2 Màn hình cột bơm xăng dầu

Màn hình cột bơm xăng dầu hiển thị các thông số sau:

- **Tổng:** Tổng số tiền ứng với số lít lượng xăng/dầu đã bơm.
- **Lít:** Số lít xăng/dầu đã bơm.
- **Đơn giá:** Số tiền trên một lít xăng/dầu

Màn hình sẽ có 3 hàng hiển thị giá trị tương ứng với cho 3 thông số trên.

Đối với hầu hết các loại cột bơm xăng dầu cũ, phổ biến trên thị trường hiện tại nói chung và với cột bơm hãng ZCheng - là đối tượng nghiên cứu của đề án - nói riêng, màn hình cột bơm gồm 3 hàng hiển thị bằng các LED 7 thanh (hình 2.2.). IC màn hình sẽ nhận dữ liệu từ bầu lường, sau đó điều khiển hiển thị từng LED.



Hình 2.2. Màn hình hiển thị cột bơm ZCheng sử dụng các LED 7 thanh

2.2 Cơ sở lý thuyết

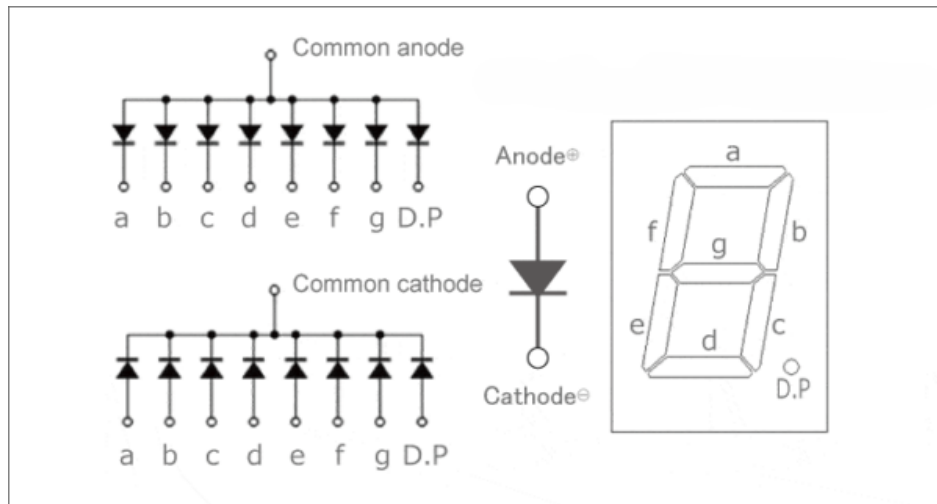
2.2.1 LED 7 thanh

[2]

LED 7 thanh (Seven-segment display) là một loại linh kiện hiển thị phổ biến dùng để hiển thị các chữ số thập phân (0–9) và một số ký tự chữ cái nhất định. Thiết bị này bao gồm 7 đoạn LED riêng biệt (ký hiệu là a, b, c, d, e, f, g), được sắp xếp theo hình dạng số 8. Mỗi đoạn là một LED nhỏ có thể phát sáng riêng biệt, cho phép tổ hợp thành các chữ số bằng cách điều khiển bật/tắt từng đoạn.

Phân loại:

- **Anode chung (Common Anode):** tất cả các cực dương (anode) của các LED được nối chung với nhau. Để bật một đoạn LED, ta cấp mức thấp (LOW) vào chân tương ứng.
- **Cathode chung (Common Cathode):** tất cả các cực âm (cathode) của các LED được nối chung với nhau. Để bật một đoạn LED, ta cấp mức cao (HIGH) vào chân tương ứng.



Hình 2.3. Cấu trúc của LED 7 thanh

Bảng mã hiển thị:

Bảng 2.1. Bảng mã nhị phân điều khiển LED 7 thanh hiển thị chữ số (Common Cathode)

Chữ số	Các đoạn sáng (abcdefg)
0	1111110
1	0110000
2	1101101
3	1111001
4	0110011
5	1011011
6	1011111
7	1110000
8	1111111
9	1111011

Ứng dụng: LED 7 thanh thường được sử dụng trong các thiết bị hiển thị đơn giản như đồng hồ điện tử, màn hình cột bơm xăng dầu, bộ đếm số, máy tính bỏ túi, mạch đo số, và các hệ thống nhúng cần hiển thị dữ liệu dạng số.

Việc điều khiển LED 7 thanh có thể thực hiện thông qua vi điều khiển bằng cách xuất tín hiệu logic ra từng chân tương ứng. Ngoài ra, có thể sử dụng IC giải mã như

74LS47 hoặc 74HC595 để giảm số chân điều khiển và đơn giản hóa mạch.

2.2.2 *Giao thức SPI*

Giao thức **SPI (Serial Peripheral Interface)** là một giao thức truyền thông nối tiếp đồng bộ do hãng Motorola đề xuất vào cuối thập niên 1980. SPI cho phép truyền dữ liệu tốc độ cao giữa vi điều khiển và các thiết bị ngoại vi như cảm biến, bộ nhớ Flash, DAC, ADC, màn hình LCD, v.v.

1. Kiến trúc tổng quát

SPI là giao thức theo mô hình **Master - Slave**, trong đó một thiết bị Master điều khiển một hoặc nhiều thiết bị Slave. Kết nối SPI sử dụng bốn đường tín hiệu chính:

- **MOSI (Master Out - Slave In)**: Dữ liệu từ Master truyền sang Slave.
- **MISO (Master In - Slave Out)**: Dữ liệu từ Slave truyền về Master.
- **SCLK (Serial Clock)**: Xung nhịp do Master tạo ra để đồng bộ dữ liệu.
- **SS (Slave Select)**: Tín hiệu chọn Slave (còn gọi là CS - Chip Select), khi mức thấp thì thiết bị Slave tương ứng được kích hoạt.

2. Cơ chế truyền dữ liệu

SPI hoạt động theo nguyên tắc truyền song song hai chiều (full duplex). Mỗi lần đồng hồ (SCLK) tạo ra một chu kỳ, dữ liệu trên MOSI và MISO đều được dịch chuyển một bit. Cả Master và Slave đều có bộ dịch (shift register), cho phép vừa gửi vừa nhận dữ liệu đồng thời.

- Truyền dữ liệu theo từng byte (8 bit) hoặc nhiều byte liên tiếp.
- Thứ tự truyền bit có thể là MSB first hoặc LSB first tùy cấu hình.
- Tần số xung đồng hồ thường trong khoảng vài trăm kHz đến hàng chục MHz.

3. Ưu và nhược điểm

Ưu điểm:

- Tốc độ truyền cao (cao hơn I²C).
- Truyền dữ liệu song công (full duplex).
- Giao thức đơn giản, dễ cài đặt.

Nhược điểm:

- Không hỗ trợ truyền đa điểm (mỗi Slave cần một chân SS riêng).
- Không có cơ chế xác thực hoặc kiểm tra lỗi tích hợp.
- Khoảng cách truyền ngắn (thường < 1m).

4. Ứng dụng

SPI được sử dụng rộng rãi trong các hệ thống nhúng có yêu cầu tốc độ cao, như:

- Giao tiếp với màn hình TFT, OLED.
- Đọc/ghi bộ nhớ Flash, EEPROM.
- Điều khiển cảm biến tốc độ cao (IMU, ADC,...).

SPI là một trong những giao thức truyền thông có dây phổ biến nhất hiện nay trong lĩnh vực điện tử nhúng, nhờ tính đơn giản, tốc độ cao và dễ triển khai.

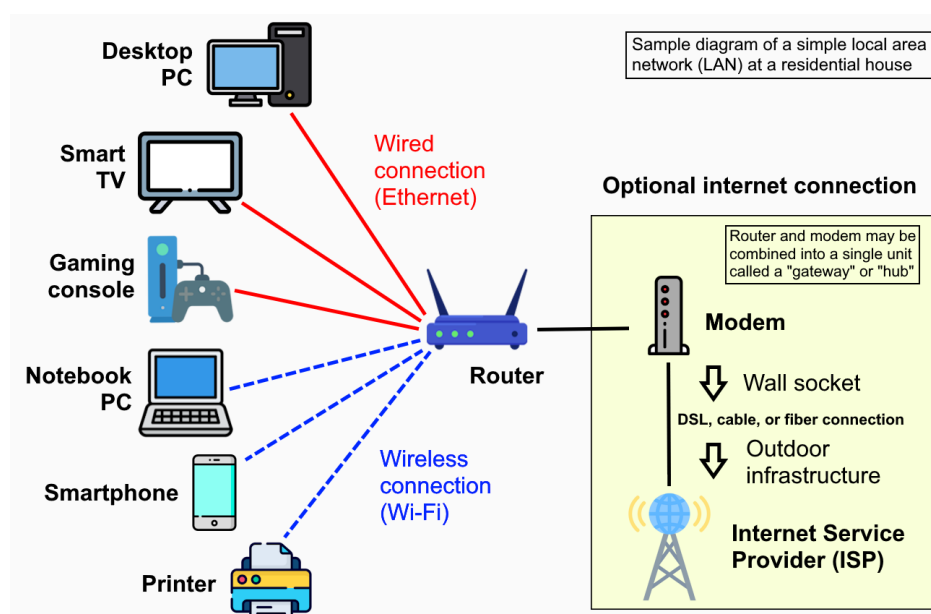
2.2.3 Mạng LAN

LAN (Local Area Network) là mạng máy tính cục bộ, cho phép nhiều thiết bị trong một khu vực địa lý nhỏ (như một tòa nhà, văn phòng hoặc nhà máy) kết nối và chia sẻ tài nguyên với nhau, chẳng hạn như dữ liệu, thiết bị ngoại vi, hoặc truy cập Internet.

1. Kiến trúc cơ bản

Một mạng LAN thường bao gồm:

- **Các thiết bị đầu cuối (end devices):** Máy tính, máy chủ, thiết bị nhúng, camera, v.v.
- **Thiết bị mạng (network devices):** Switch, router, hub, access point,...
- **Phương tiện truyền dẫn (medium):** Cáp mạng (Ethernet – Cat5e, Cat6) hoặc kết nối không dây (Wi-Fi).



Hình 2.4. Tổ chức mạng LAN

LAN sử dụng các giao thức truyền thông tầng thấp như Ethernet (IEEE 802.3) và tầng cao như TCP/IP để định tuyến và truyền dữ liệu giữa các thiết bị.

2. Ưu điểm của mạng LAN

- Tốc độ truyền dữ liệu cao (10Mbps đến 10Gbps).
- Độ trễ thấp, phù hợp với các hệ thống điều khiển thời gian thực.

- Dễ dàng mở rộng, bảo trì và quản lý.
- Chi phí triển khai thấp trong phạm vi nhỏ.

3. Ứng dụng trong hệ thống kỹ thuật LAN đóng vai trò quan trọng trong các hệ thống kỹ thuật hiện đại:

- Giao tiếp giữa thiết bị nhúng và máy chủ điều khiển.
- Hệ thống giám sát và điều khiển công nghiệp (SCADA, HMI).
- Chia sẻ dữ liệu giữa các cảm biến, bộ điều khiển và thiết bị hiển thị.

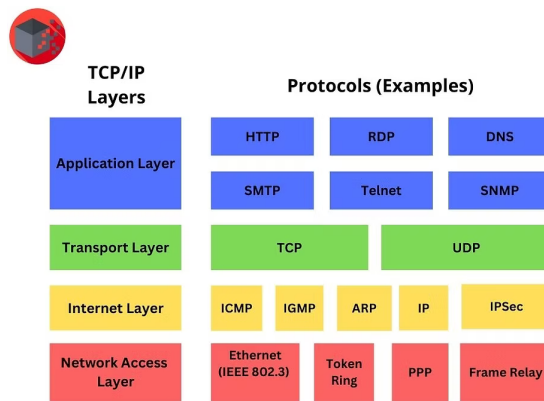
4. Các giao thức phổ biến trong LAN hỗ trợ nhiều giao thức như:

- **Ethernet (IEEE 802.3)** – Giao thức truyền thông vật lý và MAC trong hầu hết hệ thống LAN có dây.
- **ARP (Address Resolution Protocol)** – Dùng để ánh xạ địa chỉ IP sang địa chỉ MAC.
- **DHCP (Dynamic Host Configuration Protocol)** – Gán IP động cho các thiết bị trong mạng.
- **TCP/IP** – Giao thức tầng truyền tải và mạng, chịu trách nhiệm truyền dữ liệu tin cậy giữa các thiết bị.

Mạng LAN là nền tảng kết nối thiết bị trong hệ thống điện tử hiện đại, giúp các thành phần hoạt động đồng bộ, truyền dữ liệu hiệu quả và dễ dàng mở rộng quy mô hệ thống.

2.3 Kiến trúc giao thức TCP/IP

Giao thức TCP/IP (Transmission Control Protocol/Internet Protocol) là nền tảng của mạng Internet và hầu hết các mạng máy tính hiện đại. Đây là một bộ giao thức gồm nhiều lớp, được thiết kế để đảm bảo việc truyền dữ liệu giữa các thiết bị trong môi trường mạng không đồng nhất. Kiến trúc TCP/IP được phát triển bởi Bộ Quốc phòng Hoa Kỳ vào những năm 1970 và dần trở thành chuẩn thực tế trong lĩnh vực truyền thông mạng.



Hình 2.5. Kiến trúc 4 tầng của mô hình TCP/IP

Mô hình TCP/IP gồm 4 tầng:

1. **Tầng giao diện mạng (Network Interface Layer):** Còn gọi là tầng liên kết hoặc tầng truy cập mạng. Chịu trách nhiệm truyền và nhận dữ liệu qua phương tiện truyền dẫn vật lý. Tầng này tương đương với tầng Vật lý và tầng Liên kết dữ liệu trong mô hình OSI.
2. **Tầng Internet (Internet Layer):** Đảm nhiệm việc định tuyến và định địa chỉ cho các gói dữ liệu giữa các mạng. Giao thức chính tại tầng này là IP (Internet Protocol), cùng với các giao thức phụ trợ như ICMP (Internet Control Message Protocol), ARP (Address Resolution Protocol).
3. **Tầng giao vận (Transport Layer):** Cung cấp cơ chế truyền dữ liệu giữa các tiến trình ứng dụng trên các máy tính khác nhau. Hai giao thức chính là:
 - TCP (Transmission Control Protocol): Truyền dữ liệu đáng tin cậy, đảm bảo thứ tự và kiểm soát lỗi.
 - UDP (User Datagram Protocol): Truyền dữ liệu không đảm bảo, tốc độ cao, dùng trong các ứng dụng thời gian thực.
4. **Tầng ứng dụng (Application Layer):** Cung cấp giao diện trực tiếp với người dùng hoặc chương trình ứng dụng. Bao gồm các giao thức như: HTTP (truy cập web), FTP (truyền tệp), SMTP (gửi email), DNS (phân giải tên miền)...

So sánh với mô hình OSI:

Bảng 2.2. So sánh mô hình OSI và TCP/IP

Mô hình OSI	Mô hình TCP/IP
7. Application	Application
6. Presentation	
5. Session	
4. Transport	Transport
3. Network	Internet
2. Data Link	Network Interface
1. Physical	

Mô hình TCP/IP đơn giản hơn OSI và được áp dụng rộng rãi trong thực tế. Sự phân tầng rõ ràng của nó giúp việc triển khai, chuẩn hóa và khắc phục lỗi trong hệ thống mạng trở nên hiệu quả hơn.

2.3.1 Giao thức TCP

TCP (Transmission Control Protocol) là một giao thức lớp giao vận trong bộ giao thức TCP/IP, được thiết kế để cung cấp dịch vụ truyền dữ liệu tin cậy giữa các thiết bị đầu cuối trong mạng. Khác với UDP, TCP đảm bảo rằng dữ liệu đến đúng thứ tự, không bị mất mát, trùng lặp hoặc hư hỏng thông qua các cơ chế kiểm soát lỗi và điều khiển luồng.

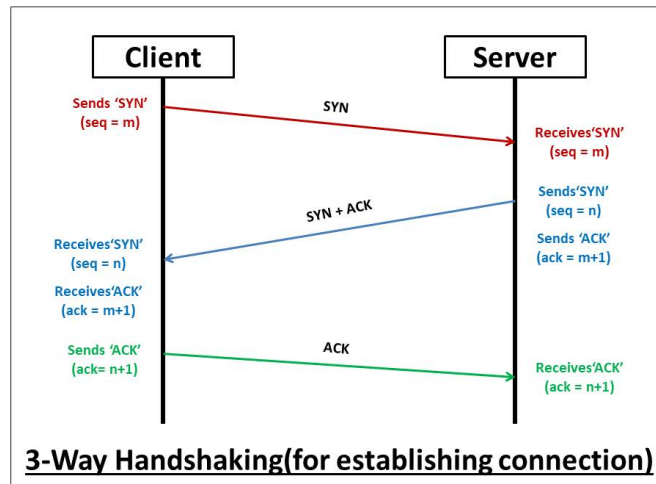
Đặc điểm chính của TCP:

- **Kết nối hướng phiên (connection-oriented):** Trước khi truyền dữ liệu, TCP thiết lập kết nối giữa hai thiết bị bằng quy trình bắt tay ba bước (three-way handshake).
- **Truyền dữ liệu đáng tin cậy:** TCP sử dụng cơ chế xác nhận (ACK), đánh số thứ tự gói tin (sequence number), kiểm tra lỗi (checksum) để đảm bảo dữ liệu truyền đến đúng đích và đúng thứ tự.
- **Kiểm soát luồng (flow control):** Điều chỉnh tốc độ truyền dữ liệu giữa hai thiết bị nhằm tránh hiện tượng nghẽn hoặc tràn bộ đệm.
- **Điều khiển tắc nghẽn (congestion control):** Giảm lưu lượng khi mạng bị quá tải, sử dụng các thuật toán như Slow Start, Congestion Avoidance, Fast Retransmit và Fast Recovery.

Quy trình bắt tay ba bước (Three-way Handshake, hình 2.6.):

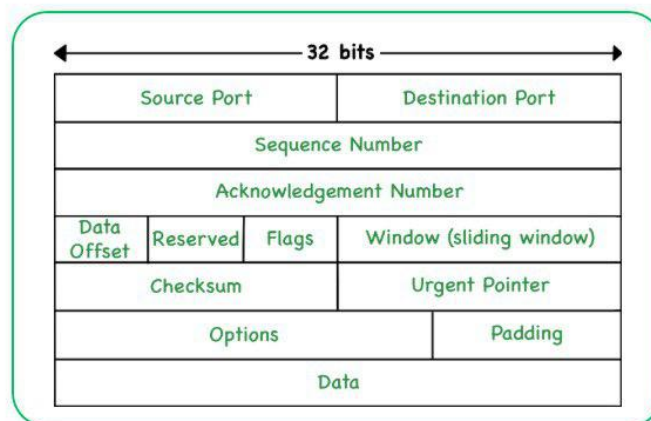
1. Client gửi một gói tin SYN đến Server để yêu cầu kết nối.

2. Server phản hồi bằng gói tin SYN-ACK để đồng ý kết nối.
3. Client gửi gói tin ACK để xác nhận, sau đó kết nối được thiết lập.



Hình 2.6. Quy trình bắt tay ba bước trong TCP

Cấu trúc gói tin TCP (hình 2.7.)



Hình 2.7. Cấu trúc gói tin TCP

- **Port nguồn / Port đích:** Xác định ứng dụng gửi và nhận.
- **Sequence Number:** Đánh dấu thứ tự byte trong dòng dữ liệu.
- **Acknowledgment Number:** Xác nhận đã nhận đến byte nào.
- **Flags:** Các cờ điều khiển như SYN, ACK, FIN...
- **Checksum:** Kiểm tra lỗi dữ liệu.

Ứng dụng của TCP: TCP được sử dụng trong các ứng dụng yêu cầu độ tin cậy cao như: trình duyệt web (HTTP/HTTPS), gửi email (SMTP, IMAP), truyền file (FTP), truy cập từ xa (SSH, Telnet)...

TCP tuy có độ trễ cao hơn UDP nhưng đổi lại cung cấp một kênh truyền thông ổn định, đáng tin cậy, phù hợp với hầu hết các ứng dụng Internet phổ biến.

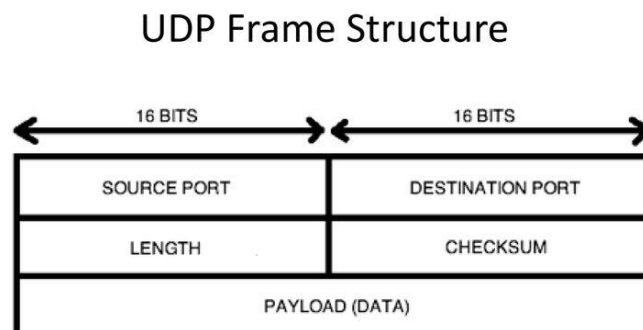
2.3.2 Giao thức UDP

UDP (User Datagram Protocol) là một giao thức ở tầng giao vận trong bộ giao thức TCP/IP, cung cấp phương thức truyền dữ liệu đơn giản, không kết nối (connectionless) và không đảm bảo độ tin cậy. UDP được sử dụng trong các ứng dụng yêu cầu tốc độ cao, truyền dữ liệu thời gian thực hoặc có cơ chế kiểm soát lỗi riêng tại tầng ứng dụng.

Đặc điểm chính của UDP:

- **Không kết nối (Connectionless):** Không cần thiết lập phiên giao tiếp trước khi gửi dữ liệu, mỗi gói tin được truyền độc lập.
- **Không đảm bảo độ tin cậy:** UDP không có cơ chế xác nhận (ACK), đánh số thứ tự hay truyền lại gói bị mất.
- **Không kiểm soát luồng hay tắc nghẽn:** Việc kiểm soát được giao cho tầng ứng dụng nếu cần.
- **Hiệu năng cao:** Do không có overhead kiểm soát phức tạp, UDP có độ trễ thấp và tốc độ truyền nhanh hơn TCP.

Cấu trúc gói tin UDP:



Hình 2.8. Cấu trúc gói tin UDP

- **Port nguồn (Source Port):** Xác định ứng dụng gửi.
- **Port đích (Destination Port):** Xác định ứng dụng nhận.
- **Độ dài (Length):** Tổng chiều dài của gói UDP (header + dữ liệu).
- **Checksum:** Kiểm tra lỗi cho phần header và dữ liệu.

Ưu điểm của UDP:

- Tốc độ truyền nhanh, độ trễ thấp.
- Phù hợp với các ứng dụng thời gian thực (real-time) như VoIP, video streaming, game online...
- Dễ dàng broadcast hoặc multicast dữ liệu đến nhiều thiết bị.

Hạn chế của UDP:

- Không đảm bảo dữ liệu được nhận hoặc nhận đúng thứ tự.
- Không có cơ chế tự động phát hiện và truyền lại khi mất gói.

Ứng dụng của UDP: UDP được sử dụng trong các ứng dụng như: DNS, DHCP, TFTP, truyền video/audio trực tiếp, gọi điện qua mạng (VoIP), hoặc các giao thức mạng yêu cầu gửi nhanh, không cần xác nhận.

Mặc dù không đảm bảo độ tin cậy như TCP, nhưng UDP là lựa chọn tối ưu cho các tình huống cần ưu tiên tốc độ và độ trễ thấp hơn là tính chính xác tuyệt đối.

2.4 Các công nghệ liên quan

2.4.1 Vi điều khiển STM32

STM32 là dòng vi điều khiển 32-bit do hãng STMicroelectronics phát triển, dựa trên kiến trúc ARM Cortex-M. Dòng STM32 rất phổ biến trong các ứng dụng nhúng (embedded systems) nhờ vào hiệu năng cao, mức tiêu thụ điện năng thấp, tích hợp nhiều ngoại vi và hỗ trợ phần mềm mạnh mẽ.

Các đặc điểm nổi bật của STM32:

- **Kiến trúc ARM Cortex-M:** STM32 sử dụng các lõi ARM như Cortex-M0, M3, M4 và M7 tùy theo dòng sản phẩm, cho phép cân bằng giữa hiệu năng và tiêu thụ điện năng.
- **Tốc độ xử lý cao:** Tốc độ xung nhịp từ 32 MHz đến hơn 400 MHz, hỗ trợ thực hiện các tác vụ thời gian thực và xử lý tín hiệu.
- **Tích hợp nhiều ngoại vi:** Bao gồm GPIO, UART, SPI, I2C, CAN, USB, ADC, DAC, PWM... hỗ trợ kết nối với nhiều loại cảm biến và thiết bị ngoại vi.
- **Đa dạng dòng sản phẩm:** STM32 được chia thành nhiều dòng như STM32F0, F1, F3, F4, F7, H7... phù hợp với nhiều mức độ ứng dụng từ đơn giản đến cao cấp.
- **Công cụ hỗ trợ phát triển mạnh mẽ:** Hỗ trợ nhiều IDE như STM32CubeIDE, Keil, IAR và thư viện phần mềm STM32Cube HAL, LL. Ngoài ra còn có công cụ cấu hình ngoại vi STM32CubeMX rất thuận tiện cho việc thiết lập ban đầu.

Ứng dụng của STM32: STM32 được sử dụng rộng rãi trong các lĩnh vực:

- Thiết bị IoT và điều khiển từ xa.
- Thiết bị đo lường và điều khiển công nghiệp.

- Thiết bị y tế, robot, và hệ thống nhúng trong ô tô.
- Thiết bị tiêu dùng như đồng hồ thông minh, máy chơi game, thiết bị điều khiển ánh sáng và âm thanh.



Hình 2.9. KIT phát triển STM32F407VET6 ARM Cortex-M4

STM32 là một nền tảng lý tưởng cho các ứng dụng nhúng nhờ khả năng mở rộng, độ ổn định cao và tài nguyên phát triển phong phú. Việc nắm vững STM32 giúp rút ngắn thời gian phát triển và tăng tính hiệu quả trong các dự án điện tử.

2.4.2 Ngôn ngữ lập trình C/C++

Ngôn ngữ lập trình C và C++ là hai ngôn ngữ phổ biến trong phát triển hệ thống nhúng và lập trình điều khiển phần cứng. Chúng cung cấp khả năng kiểm soát trực tiếp bộ nhớ, hiệu năng cao và tương thích tốt với các tài nguyên phần cứng, điều này làm cho chúng trở thành lựa chọn hàng đầu trong các ứng dụng thời gian thực như lập trình vi điều khiển STM32.

Ngôn ngữ C:

- Là ngôn ngữ lập trình thủ tục, đơn giản, dễ hiểu, và rất phù hợp với việc lập trình cấp thấp.
- Cho phép truy cập trực tiếp đến bộ nhớ thông qua con trỏ.
- Có thư viện chuẩn nhỏ gọn, dễ tích hợp vào hệ thống không có hệ điều hành.
- Là ngôn ngữ chính được sử dụng trong các trình biên dịch và thư viện hỗ trợ cho STM32 như STM32Cube HAL, CMSIS.

Ngôn ngữ C++:

- Là phần mở rộng của C, hỗ trợ lập trình hướng đối tượng, giúp tổ chức mã nguồn tốt hơn, dễ bảo trì và mở rộng.
- Cung cấp tính năng kế thừa, đa hình và đóng gói, rất hữu ích trong phát triển phần mềm phức tạp hoặc quy mô lớn.

- Cho phép sử dụng các cấu trúc hiện đại như class, template, STL (Standard Template Library).
- Ngày càng được sử dụng trong lập trình nhúng hiện đại, đặc biệt khi kết hợp với hệ điều hành thời gian thực (RTOS).

Ưu điểm khi dùng C/C++ trong lập trình vi điều khiển:

- Hiệu năng cao, thời gian thực.
- Có thể tối ưu kích thước mã nguồn và bộ nhớ sử dụng.
- Dễ dàng tích hợp với trình biên dịch như GCC, Keil, IAR...
- Được hỗ trợ rộng rãi trong cộng đồng và các tài liệu kỹ thuật.

Ứng dụng: C/C++ là công cụ chính để lập trình firmware cho vi điều khiển, viết driver điều khiển phần cứng, giao tiếp với cảm biến, mô-đun truyền thông (UART, SPI, I2C), và xây dựng các hệ thống điều khiển tự động.

2.4.3 Môi trường phát triển phần mềm NodeJS

Node.js là một môi trường chạy mã JavaScript phía máy chủ, được xây dựng trên nền tảng V8 JavaScript Engine của Google. Không giống như JavaScript truyền thống vốn chỉ chạy trên trình duyệt, Node.js cho phép thực thi mã JavaScript trong môi trường máy chủ (server-side), hỗ trợ xây dựng các ứng dụng mạng có hiệu năng cao như web server, API, công cụ dòng lệnh và hệ thống xử lý thời gian thực.

Đặc điểm nổi bật của Node.js:

- **Mô hình xử lý bất đồng bộ (asynchronous) và không chặn (non-blocking):** Node.js sử dụng vòng lặp sự kiện (event loop) và callback để xử lý nhiều yêu cầu đồng thời mà không bị treo luồng chính.
- **Một luồng (single-threaded) nhưng hiệu quả:** Thay vì tạo nhiều luồng như các nền tảng truyền thống, Node.js hoạt động hiệu quả bằng cách sử dụng cơ chế I/O bất đồng bộ để phục vụ nhiều yêu cầu cùng lúc.
- **Quản lý gói mạnh mẽ với NPM (Node Package Manager):** NPM là hệ sinh thái thư viện mã nguồn mở lớn nhất thế giới, hỗ trợ hàng triệu gói phần mềm để tích hợp vào dự án Node.js.
- **Tương thích tốt với JSON và REST API:** Node.js phù hợp để xây dựng các ứng dụng backend hiện đại kết hợp với cơ sở dữ liệu NoSQL (như MongoDB) hoặc SQL.
- **Khả năng mở rộng tốt:** Có thể mở rộng theo chiều ngang (scale-out) dễ dàng bằng cách tạo nhiều tiến trình (cluster) và phân phối tải.

Ứng dụng của Node.js:

- Xây dựng Web server và API RESTful.

- Hệ thống giám sát, điều khiển thiết bị IoT.
- Ứng dụng thời gian thực như chat, truyền phát dữ liệu (WebSocket).
- Các công cụ tự động hóa (build tool, CLI).

Công cụ hỗ trợ phát triển:

- **Visual Studio Code (VSCode):** IDE phổ biến hỗ trợ đầy đủ Node.js.
- **Express.js:** Framework web phổ biến trên nền Node.js.
- **Nodemon:** Tự động khởi động lại server khi có thay đổi mã nguồn.
- **Debug và profiling:** Hỗ trợ bởi Chrome DevTools, VSCode, hoặc các công cụ như ‘node –inspect’.

Node.js là một công nghệ lý tưởng cho các ứng dụng hiện đại yêu cầu khả năng xử lý nhiều kết nối đồng thời, phản hồi nhanh và dễ triển khai. Với cộng đồng lớn và tài nguyên phong phú, Node.js ngày càng được sử dụng rộng rãi trong phát triển phần mềm, đặc biệt là trong các hệ thống phân tán và IoT.

2.5 Lựa chọn công nghệ phù hợp

2.5.1 Chip Ethernet để kết nối với mạng LAN

Yêu cầu đặt ra là lựa chọn chip có thể kết nối Ethernet để thiết bị đọc ghi màn hình xăng dầu có thể tham gia mạng LAN, từ đó kết nối và gửi dữ liệu cho phần mềm giải mã.

Chip Ethernet cần đáp ứng nhu cầu sau:

- Giao tiếp với vi điều khiển trung tâm qua SPI do SPI có tốc độ truyền cao, có thể truyền dữ liệu liên tục không ngắt quãng.
- Hỗ trợ sẵn các giao thức: IPv4, TCP, UDP.
- Phổ biến và giá thành vừa phải.
- Ưu tiên chip có sẵn thư viện hỗ trợ bằng ngôn ngữ C/C++

Bảng 2.3. So sánh các chip Ethernet phổ biến

Tiêu chí	W5500	ENC28J60	W5100S	LAN8720A
Giao tiếp với vi điều khiển	SPI tốc độ cao (80 MHz), 8 sockets	SPI (tối đa 20MHz, phụ thuộc MCU xử lý nhiều)	SPI (nhanh và đơn giản, 70MHz), 4 sockets	RMII (cần vi điều khiển có MAC tích hợp)

Tiêu chí	W5500	ENC28J60	W5100S	LAN8720A
Hỗ trợ giao thức mạng	Tích hợp phần cứng: IPv4, TCP, UDP	Chỉ Ethernet layer (TCP/IP xử lý bằng phần mềm)	Tích hợp IPv4, TCP, UDP	Không tích hợp TCP/IP stack, chỉ là PHY
Tốc độ truyền dữ liệu	SPI 80 MHz, hiệu năng ổn định (8 Mbps TCP)	SPI 20 MHz, hiệu năng thấp (3-4 Mbps TCP)	SPI 70 MHz, cải tiến từ W5100	10/100 Mbps (phụ thuộc MAC của MCU)
Tài liệu và thư viện hỗ trợ	Rất phổ biến, Có thư viện ngôn ngữ C	Có thư viện C	Thư viện tương tự W5500, hỗ trợ tốt	Cần tích hợp LWIP hoặc stack riêng của MCU
Độ phổ biến	Phổ biến trong hệ thống nhúng hiện đại	Phổ biến trong hệ thống giá rẻ hoặc cũ	Ít phổ biến hơn W5500 nhưng cùng họ Wiznet	Thường dùng với vi điều khiển có MAC riêng (ESP32, STM32F4+)
Giá thành (tham khảo)	\$3.67	\$4.00	\$3.2 - \$3.5	\$1.03

Từ bảng liệt kê trên, chọn chip W5500 là chip Ethernet cho mạch phần cứng Thiết bị đọc ghi màn hình cây xăng.



Hình 2.10. Module Ethernet W5500

2.5.2 Vi điều khiển trung tâm cho thiết bị đọc màn hình

Yêu cầu lựa chọn:

- Có 2 cổng giao tiếp SPI: mục đích để giao tiếp với khối chuyển đổi tín hiệu và với khối Ethernet.
- Bộ nhớ flash lớn hơn 220KB (chứa được 2 firmware, mỗi firm 110kB)
- Nhanh và ổn định, dễ dàng mở rộng

Từ những yêu cầu trên , lựa chọn STM32F407VET6 là vi điều khiển xử lý trung tâm. Các tính năng nổi bật của STM32F407VET6:

- Vi điều khiển sử dụng lõi ARM Cortex-M4 32-bit, tốc độ tối đa 168 MHz, tích hợp bộ xử lý dấu chấm động (FPU) và tập lệnh DSP, phù hợp cho các ứng dụng điều khiển và xử lý tín hiệu thời gian thực.
- Bộ nhớ Flash Main Memory 512 KB, được chia thành các sector như sau:
 - Sector 0-3: mỗi sector 16 KB
 - Sector 4: 64 KB
 - Sector 5, 6 và 11: mỗi sector 128-KB

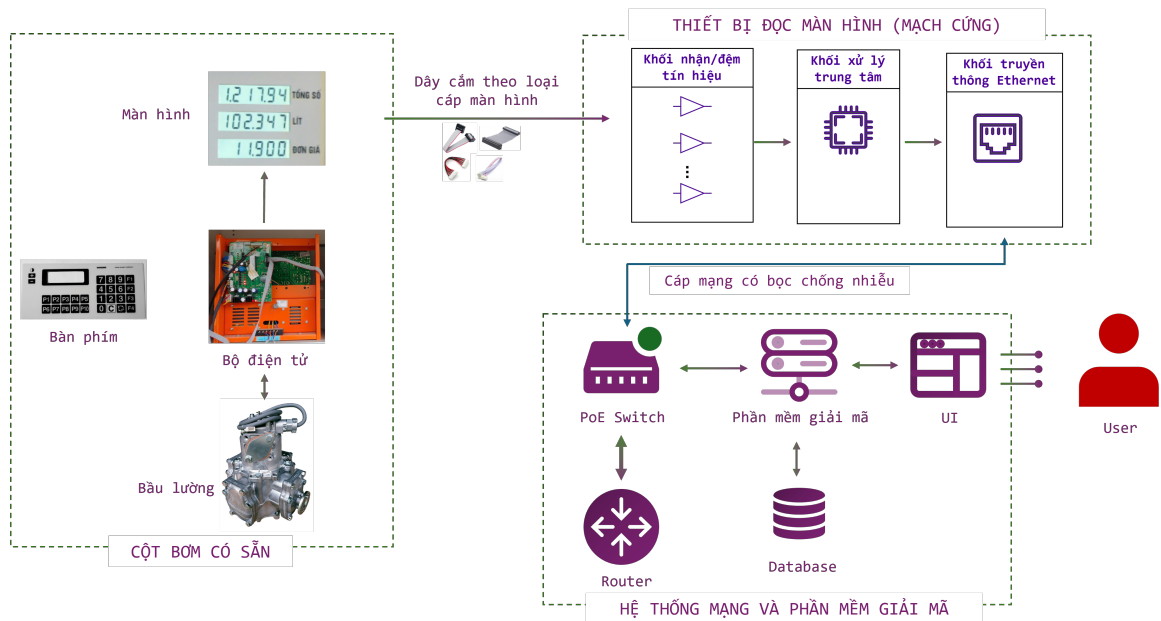
- Cho phép linh hoạt khi lưu nhiều firmware (ví dụ dual-bank firmware upgrade), hoặc phân vùng cho bootloader, data log, cấu hình...
- Tích hợp 3 cổng SPI (SPI1, SPI2, SPI3), hỗ trợ chế độ master/slave, tốc độ tối đa đến 42 MHz (SPI1) và 21 MHz (SPI2/SPI3). Phù hợp cho các giao tiếp tốc độ cao như ADC ngoại vi và module Ethernet SPI.
- Có tổng cộng 17 bộ Timer, gồm 12 timer 16-bit và 2 timer 32-bit. Hỗ trợ nhiều chế độ: PWM, encoder, input capture, output compare, rất hữu ích trong điều khiển động cơ, đo thời gian hoặc phát xung.
- Hỗ trợ nhiều chuẩn giao tiếp ngoại vi: UART, I2C, USB OTG, CAN, SDIO, Ethernet MAC - thuận tiện cho mở rộng và kết nối các module chức năng khác.
- Hoạt động với điện áp 1.8 V đến 3.6 V, hỗ trợ các chế độ tiết kiệm năng lượng như Sleep, Stop, và Standby - phù hợp cho ứng dụng nhúng có yêu cầu tiêu thụ điện thấp.

2.6 Kết luận chương

Chương này đã đưa ra những nền tảng lý thuyết chính để thiết kế thiết bị đọc ghi màn hình cột bơm xăng dầu. Đồng thời cũng đưa ra so sánh và lựa chọn linh kiện phù hợp cho 2 khối quan trọng là vi điều khiển trung tâm (STM32F407VET6) và chip giao tiếp Ethernet (W5500), làm cơ sở cho quá trình thiết kế chi tiết mạch cứng và phần mềm, sẽ được trình bày ở Chương 3.

CHƯƠNG 3. THIẾT KẾ HỆ THỐNG

3.1 Phân tích yêu cầu hệ thống



Hình 3.1. Các thành phần của hệ thống

Các thành phần chính của hệ thống bao gồm:

3.1.1 Thiết bị đọc ghi màn hình

Là mạch cứng gắn vào màn hình cột bơm, thu dữ liệu thô màn hình:

- Kích cỡ: vừa, có thể lắp đặt trong cột bơm
- Tần số lấy mẫu: 100Hz
- Cấp nguồn riêng
- Nguồn và tín hiệu vào được cách ly quang, đảm bảo không có tín hiệu quay ngược trở lại màn hình
- Có 2 relay đóng ngắt có thể nối nối tiếp với cột bơm
- Giao tiếp, gửi dữ liệu thông qua Ethernet
- (Optional) Đầu vào tín hiệu (cáp và bộ chuyển đổi tín hiệu) có thể thích hợp với nhiều loại màn hình khác ngoài ZCheng để phục vụ thu thập và phân tích thêm các loại cột bơm khác.

3.1.2 Phần mềm giải mã

Phần mềm giải mã đặt trong máy tính nội bộ tại trạm:

- Đọc và giải mã tín hiệu tới màn hình với tần số 100Hz.

- Giao diện phần mềm hiển thị màn hình đã giải mã theo thời gian thực
- Phát hiện, lưu trữ các phiên bơm. Giao diện hiển thị các phiên bơm đã lưu trữ.
- Điều khiển đóng ngắt relay.
- Có cơ chế cập nhật firmware (OTA) cho thiết bị.

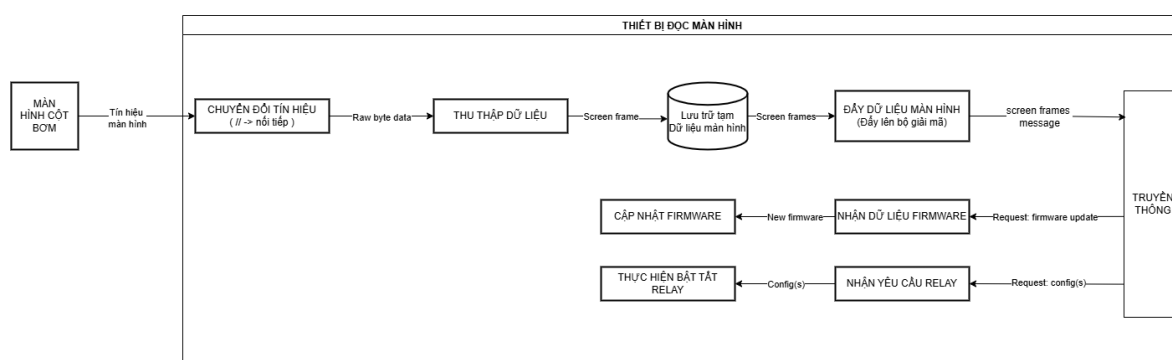
3.1.3 Hệ thống mạng

- POE Switch cấp nguồn riêng cho thiết bị đọc ghi màn hình
- Router điều khiển mạng LAN, cấp phát IP cho thiết bị
- Các thiết bị (thiết bị đọc ghi màn hình và phần mềm giải mã trong máy tính nội bộ) có thể tự động dò tìm và kết nối với nhau.

3.2 Mô hình thiết kế tổng thể

Phần này nêu ra sơ đồ khối chức năng các thành phần hệ thống. Qua đó mô tả chức năng cụ thể của thành phần thông qua các khối, đồng thời mô tả cách các khối giao tiếp với nhau.

3.2.1 Sơ đồ khối chức năng thiết bị đọc ghi màn hình (Device)



Hình 3.2. Sơ đồ khối chức năng thiết bị đọc ghi màn hình (Device)

Giải thích các khối chức năng:

• Chuyển đổi tín hiệu:

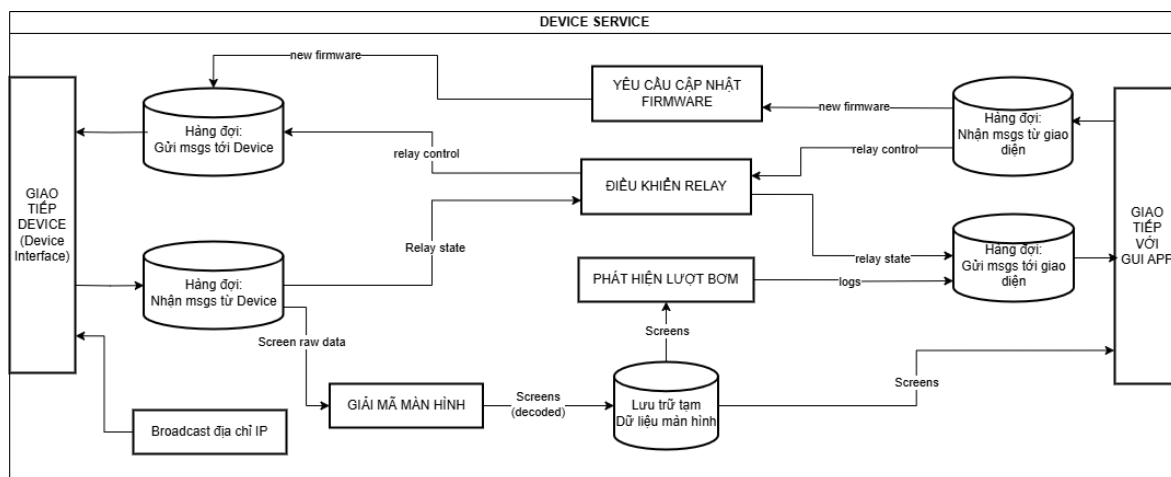
Nếu tín hiệu gửi tới màn hình cột bơm là nối tiếp, khối này đệm tín hiệu nối tiếp tới bộ Thu thập dữ liệu

Nếu tín hiệu gửi tới màn hình cột bơm là song song, có nhiều chân dữ liệu, khối này chuyển đổi tín hiệu từ các chân thành tín hiệu nối tiếp -> chuyển đổi thành các SPI frame để đệm tới bộ Thu thập dữ liệu.

- **Thu thập dữ liệu:** Thu thập các byte data tổng hợp thành các screen frame, lưu tạm vào 1 in-mem database.
- **Đẩy dữ liệu lên màn hình:** Thiết bị đọc màn hình tạo 1 message (request message) gồm nhiều screen frame để đẩy lên máy tính local

- **Nhận và cập nhật firmware mới:** Thiết bị đọc màn hình có thể nhận firmware mới từ máy tính local hoặc Logi Service và tự động cập nhật, khởi động lại
- **Nhận và cập nhật relay:** Thiết bị đọc ghi màn hình có thể nhận và thực hiện yêu cầu bật tắt relay
- **Truyền thông:** Lấy địa chỉ IP, dò tìm địa chỉ của phần mềm giải mã và tự động kết nối. Truyền nhận dữ liệu với phần mềm giải mã thông qua mạng LAN.

3.2.2 Sơ đồ khối chức năng phần mềm giải mã (Device Service)



Hình 3.3. Sơ đồ khối chức năng phần mềm giải mã và chốt phiên bơm (Device Service)

Giải thích các khối chức năng:

- **Giao tiếp với Device (Thiết bị đọc ghi màn hình):**

Khối này được thiết kế chạy trên 1 luồng độc lập, nghe các yêu cầu kết nối từ thiết bị đọc ghi màn hình, kiểm tra và giữ kết nối nếu thiết bị hợp lệ

Truyền nhận gói tin với thiết bị đọc ghi và đẩy dữ liệu tới các khối khác để xử lý logic chính.

- **Giao tiếp với giao diện (GUI App):** Giữ kết nối và truyền nhận dữ liệu với phần mềm giao diện bao gồm:

Stream dữ liệu màn hình đã giải mã theo thời gian thực.

Gửi các phiên bơm (log) phát hiện được và trạng thái relay.

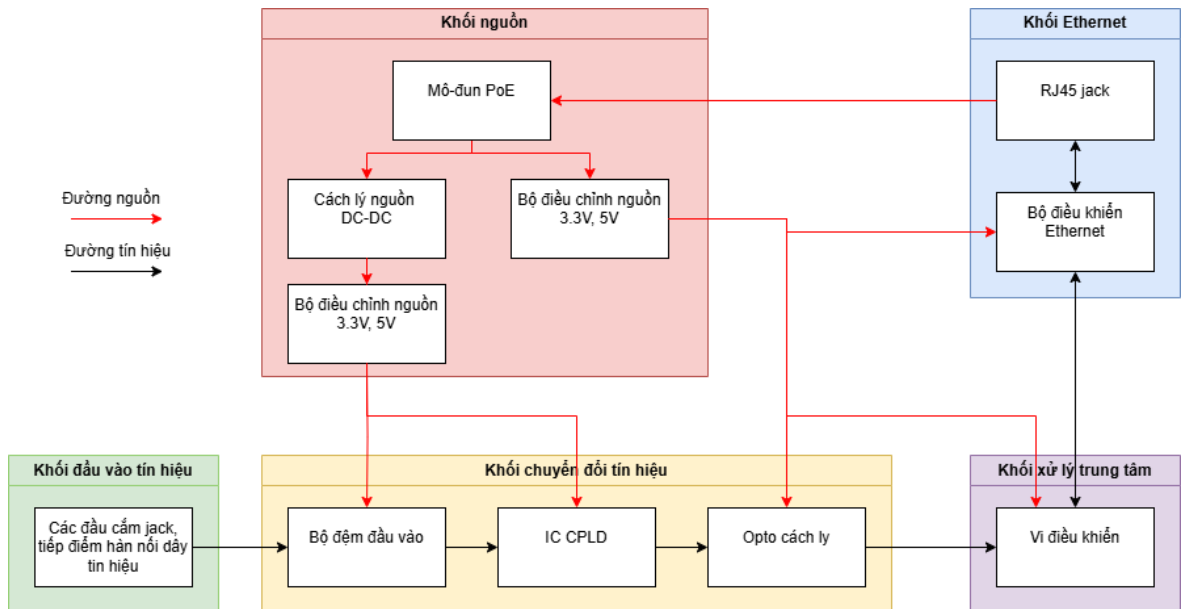
Nhận các yêu cầu từ giao diện: cập nhật firmware, relay.

- **Giải mã màn hình và phát hiện phiên bơm:** Giải mã các gói tin dạng byte theo từng loại màn hình để được dữ liệu màn hình thời gian thực (số tiền, lít, đơn giá). Từ đó phát hiện trạng thái của máy (đang dừng và dừng trong bao nhiêu giây, đang tăng, reset) và phát hiện phiên bơm.
- **Cập nhật firmware:** Nhận file firmware từ giao diện, kiểm tra và thực hiện OTA với thiết bị đọc màn hình

- **Điều khiển relay:** Chuyển tiếp gói tin yêu cầu bật tắt relay tới thiết bị đọc ghi màn hình

3.3 Thiết kế phần cứng thiết bị đọc màn hình

3.3.1 Sơ đồ khối phần cứng thiết bị đọc màn hình



Hình 3.4. Sơ đồ khối triển khai phần cứng thiết bị đọc ghi màn hình

Thiết bị mạch cứng được cấp nguồn riêng bởi POE Switch. Nguồn và tín hiệu vào được các ly bởi bộ cách ly nguồn DC-DC, các bộ đệm đầu vào và Opto cách ly, đảm bảo không ảnh hưởng tới nguồn cấp của cột bơm cũng như không có tín hiệu hay nhiễu đẩy ngược về cột bơm.

Khối đầu vào tín hiệu và khối chuyển đổi tín hiệu cũng được thiết kế thêm để thu và khảo sát được nhiều loại màn hình khác, phục vụ cho việc mở rộng chức năng hệ thống.

Thiết bị giao tiếp với phần mềm giải mã thông qua mạng LAN, sử dụng khối Ethernet.

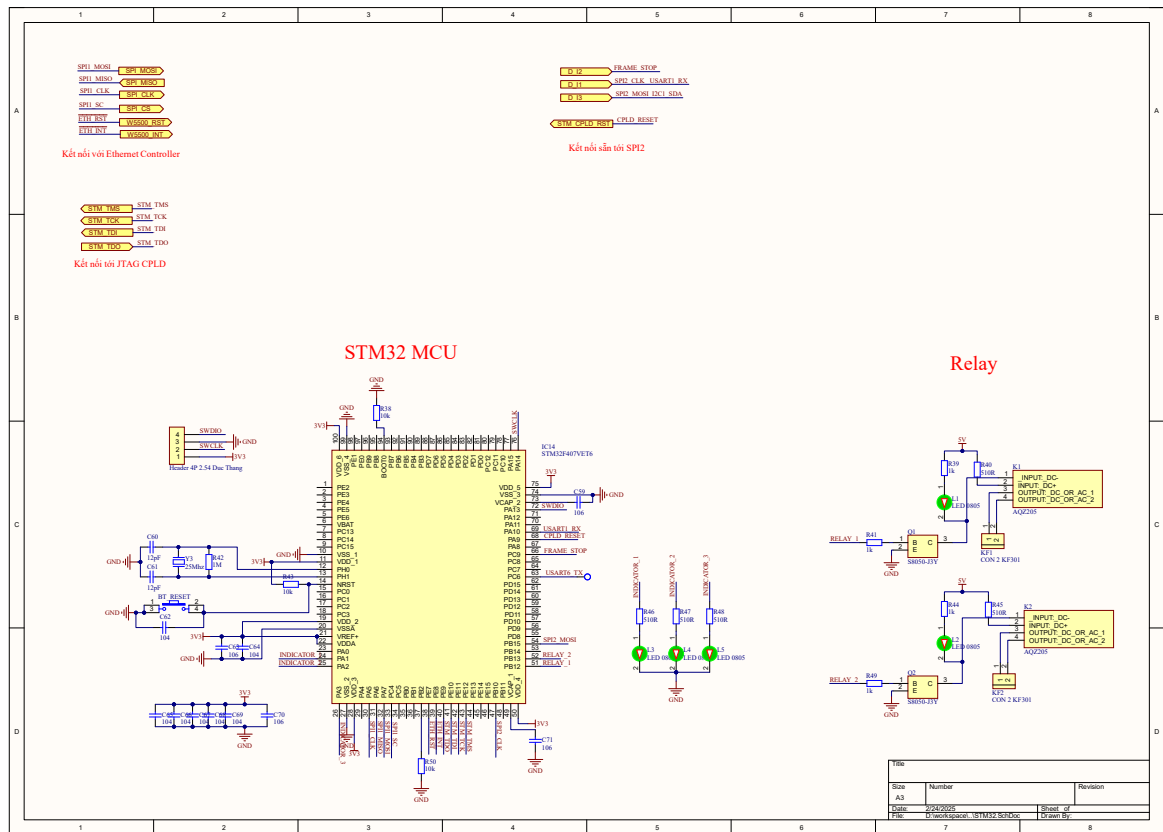
3.3.2 Lựa chọn linh kiện và vẽ sơ đồ nguyên lý

a. Khối xử lý trung tâm

Từ những yêu cầu đã nêu ở Chương 2, lựa chọn STM32F407VET6 là vi điều khiển xử lý trung tâm STM32F407VET6:

Sơ đồ nguyên lý mạch:

- Nguồn cấp 3.3V được đưa vào các chân VDD và Vref của module, các chân VSS được nối đất. Thiết kế nguồn nối với đất qua các tụ Bypass có nhiệm vụ lọc nhiễu cao tần cho nguồn nuôi vi điều khiển.
- Sử dụng SPI1 để giao tiếp với chip Ethernet. SPI2 để giao tiếp với khối chuyển đổi dữ liệu



Hình 3.5. Sơ đồ nguyên lý khối xử lý trung tâm

- Các chân PA1, PA2, PA3 điều khiển đèn báo led
- Reset STM32 được thực hiện khi chân NRST hạ xuống mức 0:
 - Điện trở kéo lên 10KΩ
 - Tụ điện: $C = 100\text{nF}$, giúp tạo xung reset ngắn nhưng đủ để tái khởi STM32
- Mạch nạp cho STM32:
 - Thiết kế mạch nạp theo chuẩn SWD
 - Hỗ trợ nạp thông qua mạch nạp STLink

Khởi đầu vào tín hiệu

Yêu cầu lựa chọn:

Ngoài việc thu và giải mã loại màn hình ZCheng, khởi đầu vào tín hiệu còn cần hỗ trợ các loại đầu vào cho màn hình khác để thu mẫu dữ liệu, phục vụ cho việc giải mã, mở rộng hệ thống sau này. Các loại đầu vào màn hình đã khảo sát được bao gồm:

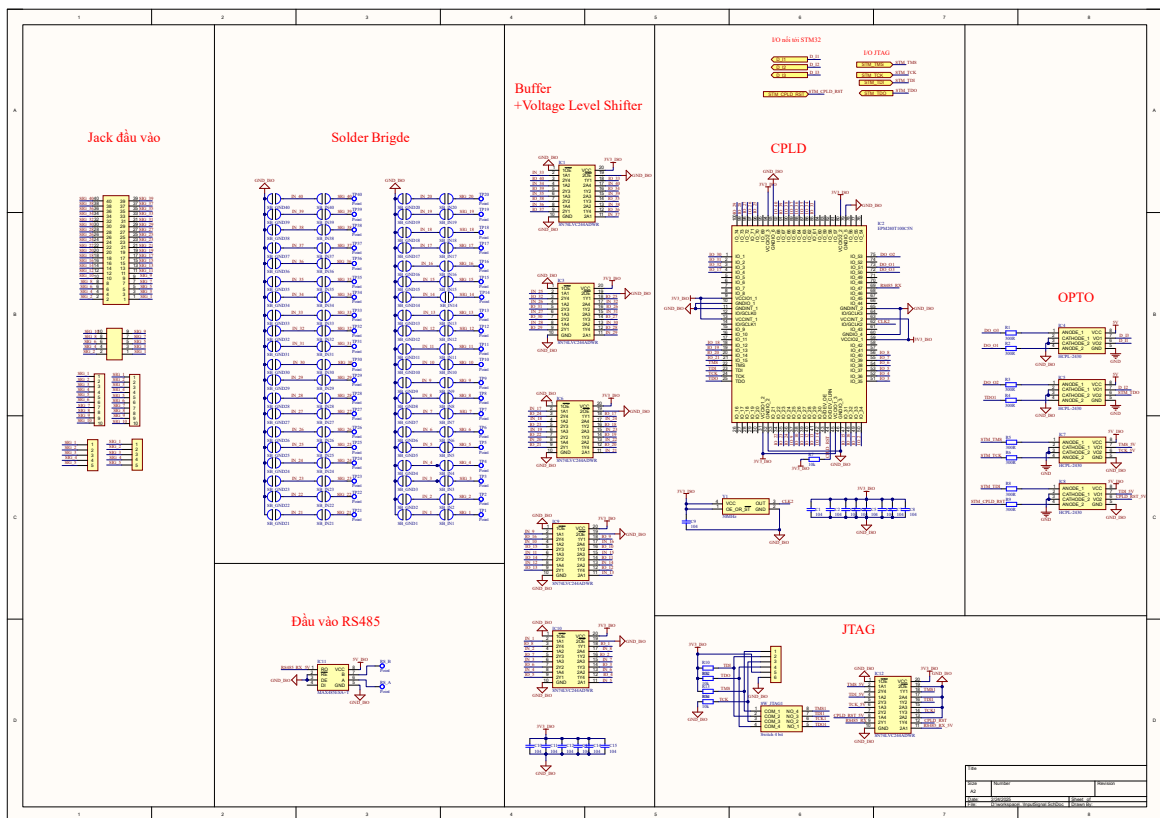
- Đầu vào SPI (cho màn hình ZCheng, đã giải mã được). Loại cáp: 8P 3.96mm. Số chân tín hiệu: 3

- Loại cáp: 5P 3.96mm, số chân tín hiệu: 2
- Loại cáp: IDE 10. Số chân tín hiệu 3
- Loại cáp: IDE 40. Số chân tín hiệu 30

Với danh sách loại đầu vào ở trên, thiết kế khối đầu vào tín hiệu gồm các đầu jack cắm và tiếp điểm hàn để đấu nối dây điện. Khối gồm 2 phần:

- Các đầu cắm cáp: hỗ trợ các loại cáp đã khảo sát.
- Các điểm hàn nối dây: Trường hợp các loại màn hình khác, sẽ hàn các đường tín hiệu đó thông qua các đầu tiếp điểm hàn, kèm dây điện nối (nếu cần). Vị trí mỗi hàn, đấu dây sẽ cố định theo từng loại tín hiệu màn hình.

Tổng số chân cho khối đầu vào tín hiệu là 40 chân.



Hình 3.6. Sơ đồ nguyên lý khối đầu vào tín hiệu

Khối nguồn và Ethernet

Yêu cầu thiết kế:

- Khối nguồn:
 - Cấp nguồn 3.3V cho khối khác. Chuyển đổi nguồn 5V của POE sang 3.3V.

– Cách ly và chống nhiễu.

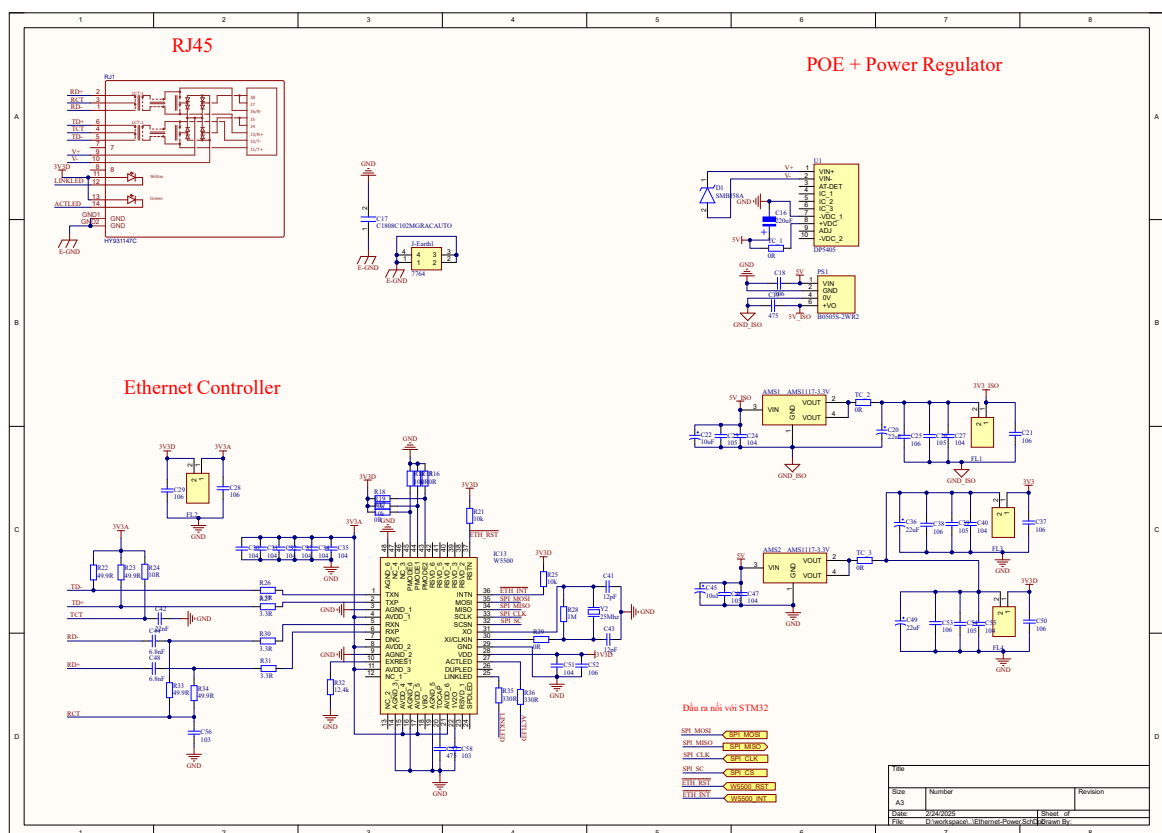
- Khởi Ethernet:

– Chống nhiễu.

– Giao tiếp với MCU (STM32) theo chuẩn SPI.

Với những yêu cầu trên, lựa chọn:

- B0505S-2WR2 và AMS1117-3.3V để cách ly và chuyển đổi nguồn POE thành nguồn 3.3V.
- Chip W5500 để giao tiếp Ethernet.



Hình 3.7. Sơ đồ nguyên lý khối nguồn và giao tiếp Ethernet

Giải thích khối nguồn:

- Mô-đun DP5405 để nhận nguồn từ PoE Switch (thông qua jack RJ45), chuyển đổi sang điện áp 5VDC để cấp nguồn cho toàn bộ thiết bị.
- IC B0505S: Có chức năng chuyển đổi điện áp 5VDC sang điện áp 5VDC cách ly. Việc cách ly nguồn để góp phần cách ly tín hiệu điện giữa tín hiệu của cáp màn hình với tín hiệu khác trong mạch, để 2 loại tín hiệu này không ảnh hưởng tới nhau.

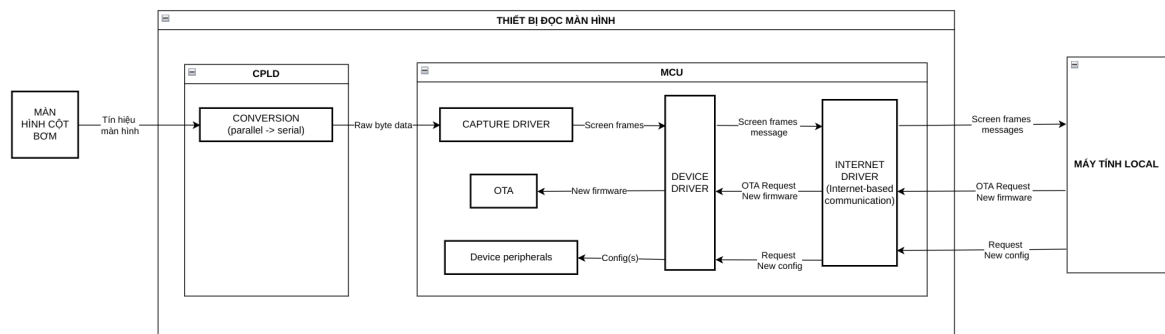
- IC AMS1117-3.3V: Là IC điều chỉnh nguồn, đầu ra IC sẽ cung cấp nguồn điện áp 3.3V cho các linh kiện khác trong mạch. Có 2 IC trong đó IC thứ nhất để chuyển đổi điện áp 5VDC sang 3.3VDC, IC thứ hai chuyển đổi điện áp 5VDC cách ly sang 3.3V, từ đó điện áp 3.3V này cũng được cách ly với điện áp 3.3VDC ở IC thứ nhất.

Giải thích khối Ethernet:

- Jack cắm Ethernet: Sử dụng jack RJ45 HY931147C để cắm dây Ethernet vào thiết bị, vừa để dẫn nguồn cho thiết bị, vừa để hỗ trợ giao tiếp mạng Ethernet.
- IC điều khiển Ethernet: Sử dụng IC W5500 để cung cấp các chức năng liên quan đến truyền/nhận dữ liệu thông qua Ethernet cho khối Xử lý trung tâm.

3.4 Thiết kế firmware thiết bị đọc màn hình

3.4.1 Kiến trúc triển khai firmware



Hình 3.8. Sơ đồ khối triển khai firmware cho thiết bị

Các khối triển khai thực hiện các chức năng đã được nêu ra trong hình 3.2.

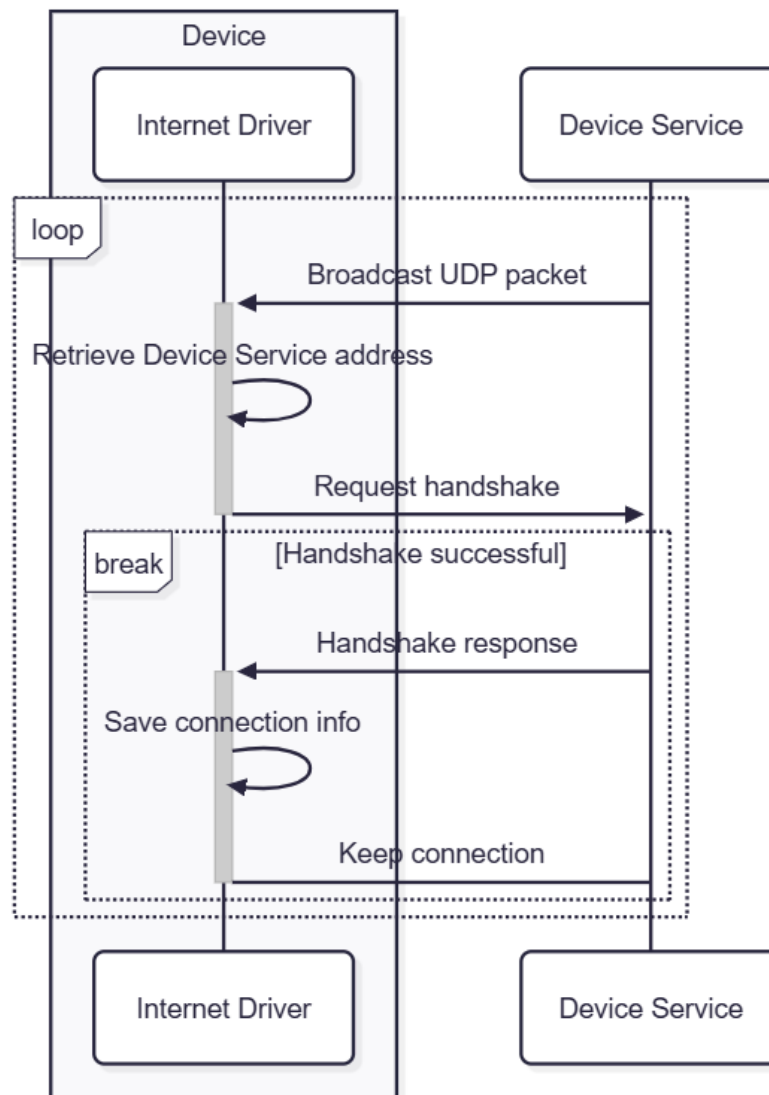
Mỗi khối triển khai ở hình 3.8. được tổ chức thành một module mã nguồn, xử lý và trao đổi dữ liệu với nhau.

- Capture Driver: Nhận các SPI frame từ CPLD, đóng gói thành các dữ liệu màn hình dạng byte data, cùng thời gian nhận dữ liệu và loại màn hình.
- Device Driver: Xử lý giao thức. Xử lý các gói tin nhận được (dạng byte frame) từ đầu vào Ethernet và chuyển tiếp tới các khối Capture Driver, OTA và Device peripherals để thực hiện các logic chính. Đồng thời đẩy các gói tin màn hình (dạng byte frame) cho khối Internet để thực hiện gửi cho phần mềm máy tính (Device Service)
- Internet Driver: Giao tiếp với chip W5500 để thực hiện trao đổi gói tin qua mạng LAN.
 - Xin cấp phát IP tĩnh từ router
 - Nhận địa chỉ IP của phần mềm máy tính (Device Service) thông qua gói tin được broadcast

- Trao đổi dữ liệu dạng byte frame với phần mềm máy tính (Device Service) thông qua giao thức TCP, cấu trúc gói tin tuân theo API đã được quy ước.
- Khi bị mất kết nối với phần mềm Device Service, cố gắng kết nối trở lại với phần mềm.

3.4.2 Trình tự giao tiếp giữa các khối

a. Nhận gói tin broadcast và kết nối tới phần mềm Device Service

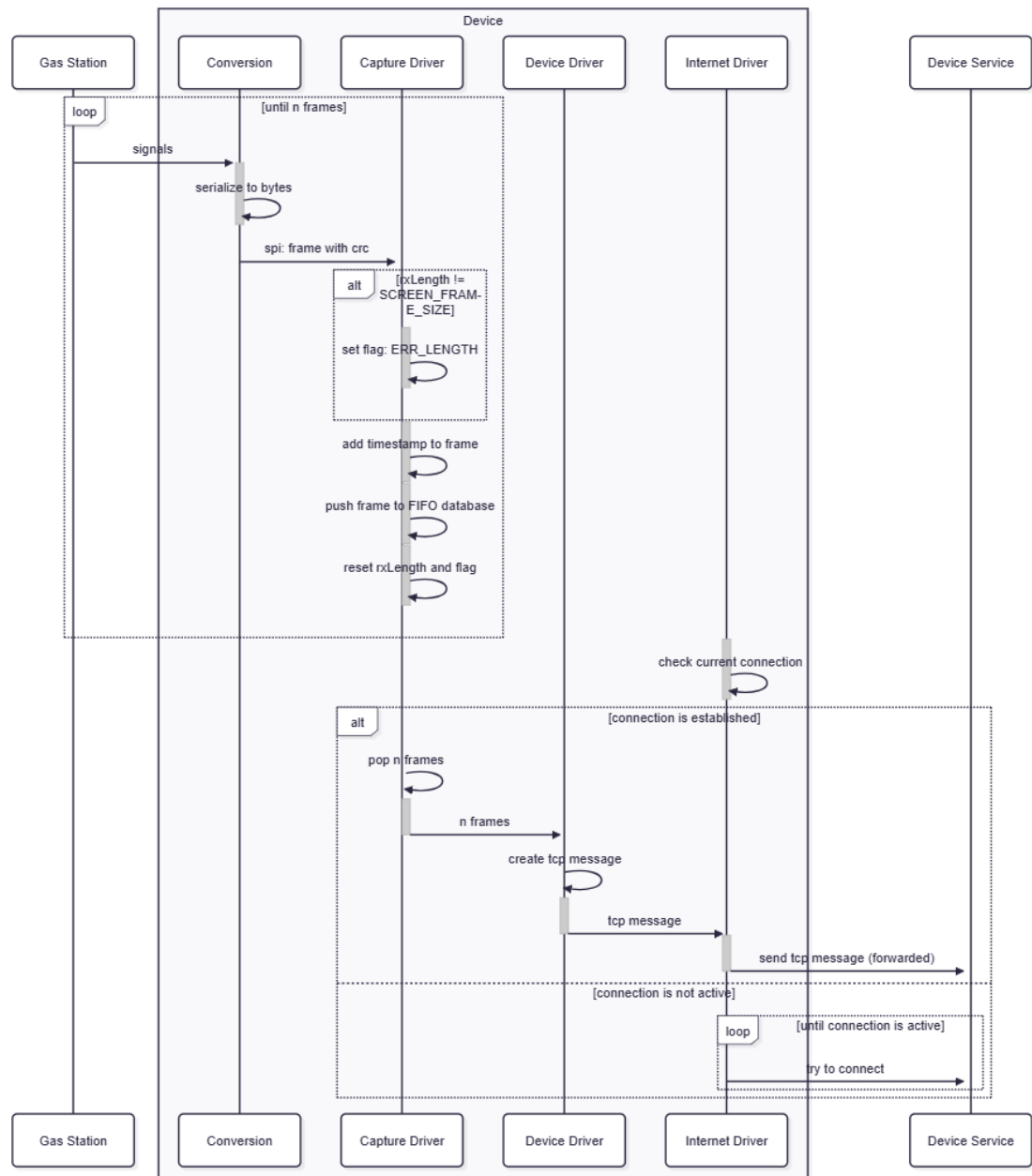


Hình 3.9. Trình tự giao tiếp: Thiết lập kết nối tới phần mềm giải mã Device Service

Gói tin broadcast của phần mềm giải mã (Device Service) có chứa địa chỉ (IP và PORT) của phần mềm. Thiết bị nhận được gói tin, lấy thông tin địa chỉ và gửi yêu cầu thiết lập kết nối tới phần mềm (handshake).

Trong trường hợp không kết nối được hoặc mất kết nối, thiết bị sẽ liên tục yêu cầu kết nối trở lại tới phần mềm.

b. Đẩy dữ liệu thô màn hình (dạng byte)



Hình 3.10. Trình tự giao tiếp: thu thập và đẩy dữ liệu màn hình

Tín hiệu màn hình được thu thập bởi bộ chuyển đổi tín hiệu, và đẩy tới **Capture Driver**, sử dụng SPI.

Khối Capture Driver nhận từng byte, lưu trữ tạm vào 1 buffer cho đến khi độ dài buffer bằng độ dài tiêu chuẩn 1 frame màn hình (22 byte đối với máy ZCheng). Khi

đó, dữ liệu buffer được lấy ra để tạo thành 1 frame màn hình.

Mỗi frame màn hình chứa dữ liệu "tổng tiền", "số lít", "đơn giá" đã được mã hóa, sẽ được lưu tạm vào 1 FIFO database (tại memory) cùng với thời gian nhận frame. Buffer nhận sau đó được xóa để bắt đầu nhận frame mới.

Do tần số nhận frame cao (100Hz), không thể gửi dữ liệu với tần số như vậy thông qua chip Ethernet, dữ liệu màn hình được lưu tạm vào 1 FIFO Database. Ở mỗi lần gửi, **Internet Driver** kiểm tra kết nối hiện tại với phần mềm Device Service. Nếu kết nối vẫn tốt, **Capture Driver** sẽ lấy nhiều frame từ database cùng một lúc để gửi đến phần mềm Device Service.

c. Cập nhật firmware

Phần mềm Device Service gửi yêu cầu bắt đầu tiến trình OTA cùng với kích thước firmware mới. Thiết bị thông báo lại tên file firmware muốn cập nhật (file 1 hoặc 2).

Sau đó, Device Service gửi từng đoạn dữ liệu firmware mới cùng offset. Thiết bị dữ liệu vào flash, tại vị trí cho firmware mới.

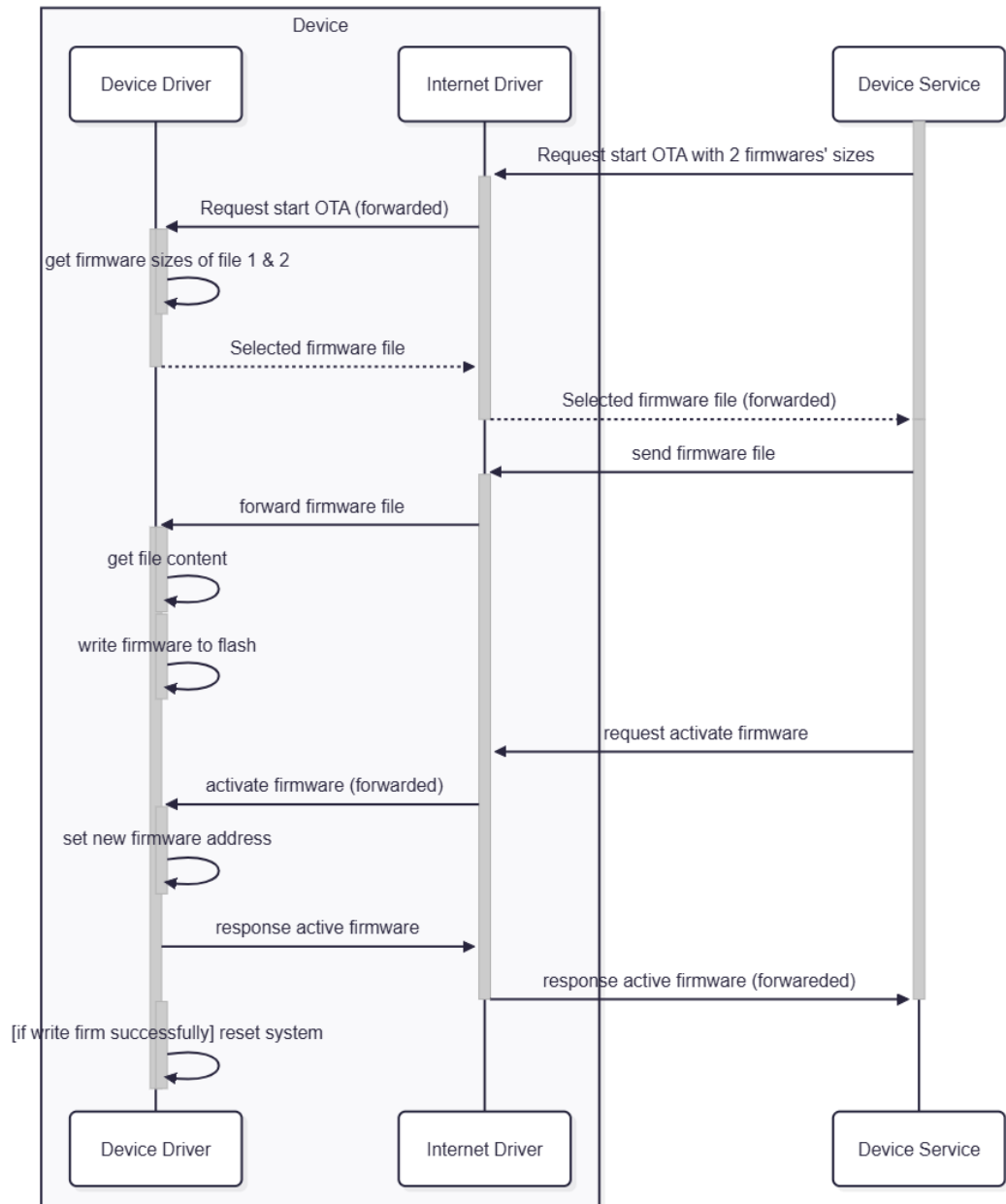
Khi việc truyền nhận firmware mới hoàn tất, thiết bị đặt con trỏ chứa địa chỉ firmware theo giá trị địa chỉ mới và tiến hành reset.

3.5 Thiết kế phần mềm giải mã Device Service

3.5.1 Kiến trúc triển khai phần mềm

Các khối triển khai thực hiện các chức năng đã nêu ở sơ đồ khối chức năng (hình 3.3.). Về mặt triển khai, Device Service là phần mềm đa luồng, chạy dưới dạng tiến trình chạy ngầm trên máy tính nội bộ, trong đó có các luồng chính:

- Device Communication giữ kết nối và giao tiếp và điều khiển thiết bị đọc ghi màn hình (Device), thực hiện giải mã và chốt phiên bơm.
- Client Event: luồng này trao đổi các sự kiện với phần mềm giao diện bao gồm:
 - Các phiên bơm (gọi là log).
 - Yêu cầu bật tắt relay (từ phía giao diện) và thông báo trạng thái relay (từ phía thiết bị) cho giao diện.
 - Yêu cầu cập nhật firmware và nội dung file firmware.
- Client Raw: luồng riêng biệt để gửi trực tiếp các màn hình đã giải mã theo thời gian thực, để giao diện có thể hiển thị giá trị tương ứng với màn hình cây xăng thực tế.
- Broadcast: luồng độc lập để broadcast gói tin UDP chứa địa chỉ (IP và PORT) của Device Service này cho toàn bộ thiết bị trong mạng LAN. Các thiết bị đọc ghi màn hình nhận được địa chỉ và chủ động kết nối.



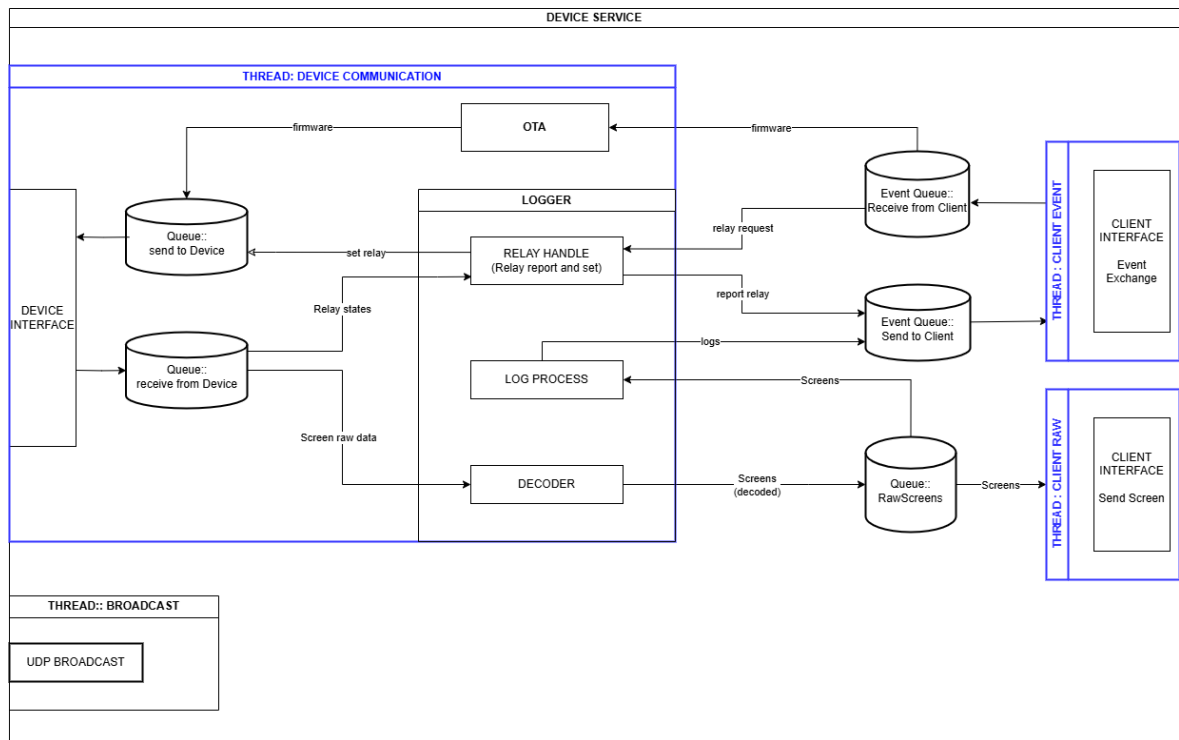
Hình 3.11. Trình tự giao tiếp: cập nhật firmware từ xa (OTA)

Các luồng giao tiếp với nhau thông qua các hàng đợi (Queue). Các hàng đợi này được thiết kế để thread-safe, tức là các luồng chạy song song có thể truy cập vào hàng đợi mà không xảy ra xung đột.

3.5.2 Trình tự giao tiếp và thuật toán cho các khối

Phần này trình bày trình tự giao tiếp giữa các khối trong các use case cụ thể, đồng thời trình bày các lưu đồ thuật toán quan trọng. Các use case chính bao gồm:

- Giải mã và gửi giá trị màn hình theo thời gian thực



Hình 3.12. Sơ đồ khối triển khai phần mềm Device Service

- Cập nhật firmware cho thiết bị
- Gửi yêu cầu đóng ngắt relay

a. Giải mã, gửi giá trị màn hình và phát hiện phiên bơm

Việc giải mã tiến hành qua các bước sau:

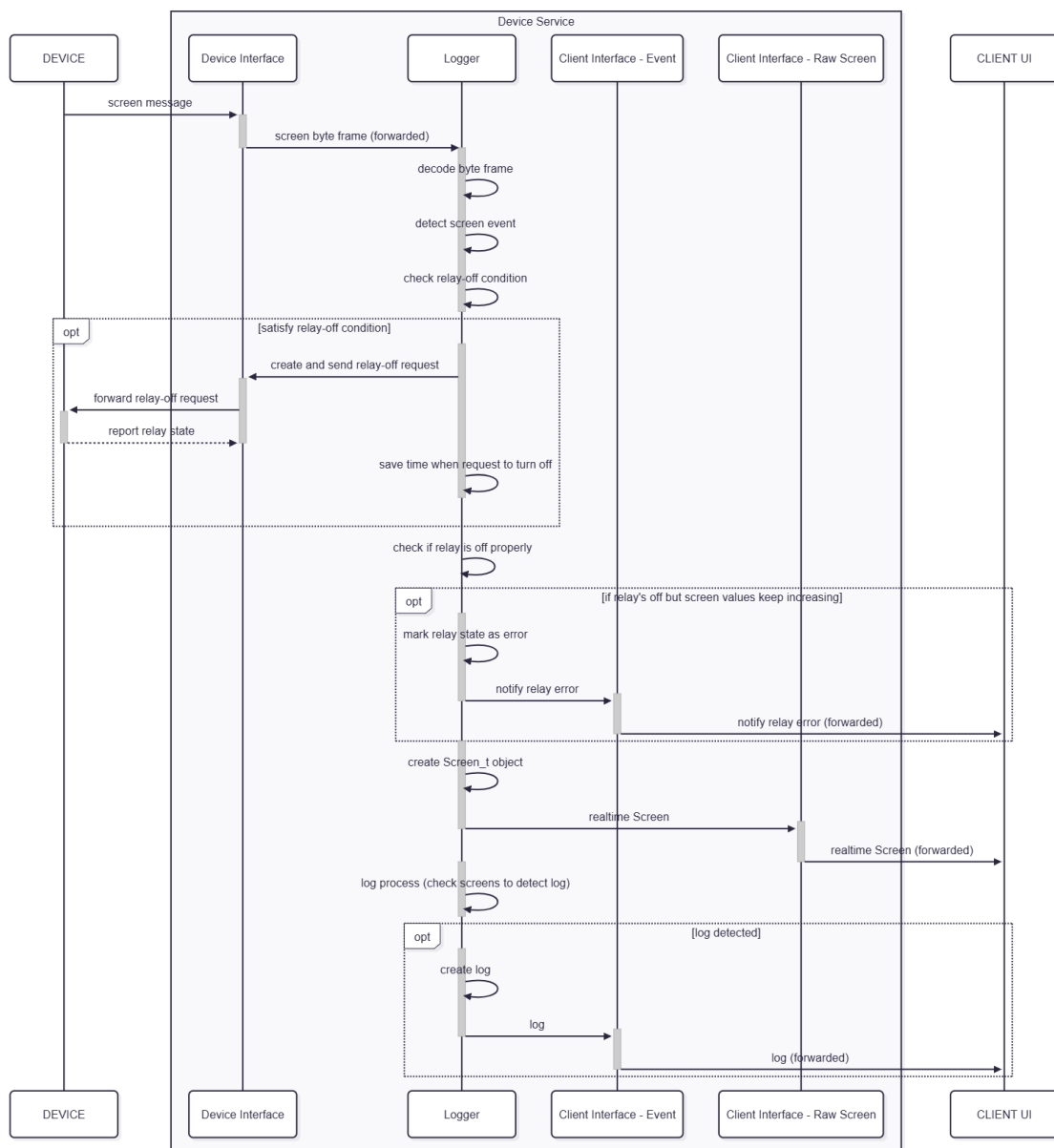
- Giải mã dữ liệu thô dạng byte thành giá trị màn hình (số tiền, số lít, đơn giá).
- Phân loại sự kiện màn hình (screen event):
- Phát hiện trạng thái máy (machine state).
- Phát hiện phiên bơm (log) nếu có.

Dữ liệu thô từ màn hình cây xăng được đọc liên tục, decode và so sánh để xác định giá trị và độ biến thiên giá trị đang hiển thị ở cột bơm ('bằng 0 | đang tăng | đang dừng'), xác định trạng thái hiện tại của cột bơm ('reset | đang bơm | tạm dừng bơm'). Từ đó đưa ra quyết định chốt một lượt bơm - gọi là 1 'log'.

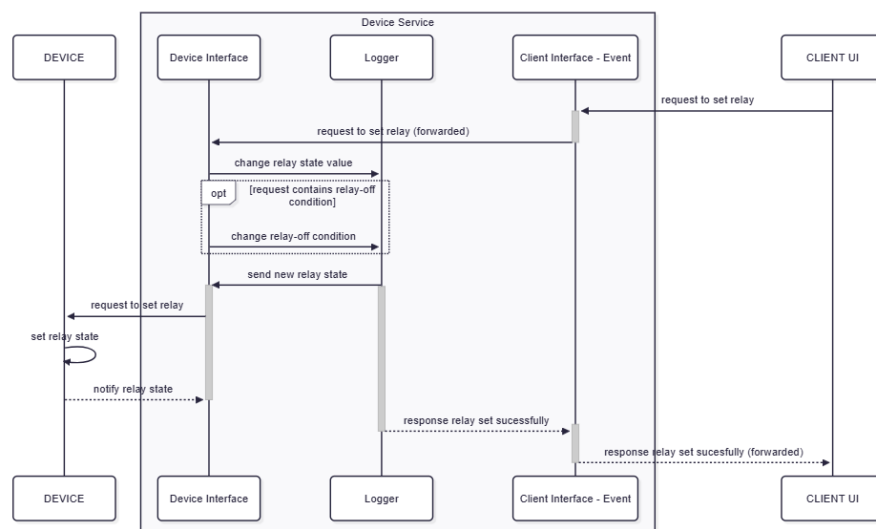
b. Điều khiển relay cho thiết bị

Từ giao diện người dùng Client UI, có thể:

- Gửi yêu cầu bật tắt relay.
- Đặt ngưỡng giới hạn trên (số tiền hoặc số lít) để relay tự động tắt.



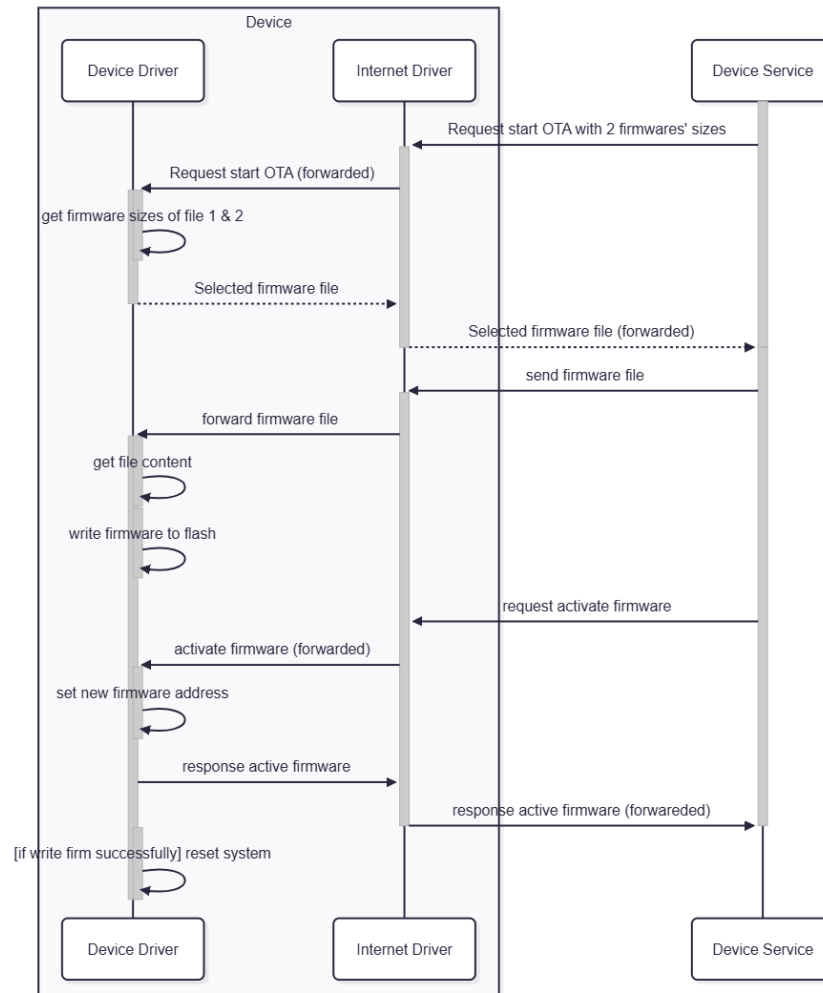
Hình 3.13. Trình tự giao tiếp cho phần mềm giải mã: Giải mã màn hình



Hình 3.14. Trình tự giao tiếp phần mềm trong trường hợp điều khiển relay

c. Cập nhật firmware từ xa (OTA)

File firmware mới được gửi từ Client UI, được mã hóa dưới dạng base64 string. Phần mềm Device Service giải mã và kiểm tra hợp lệ (verify hash) rồi lưu tạm vào RAM. Sau khi nhận được yêu cầu kích hoạt quá trình OTA, Device Service bắt đầu gửi từng đoạn firmware mới tới thiết bị đọc ghi màn hình.



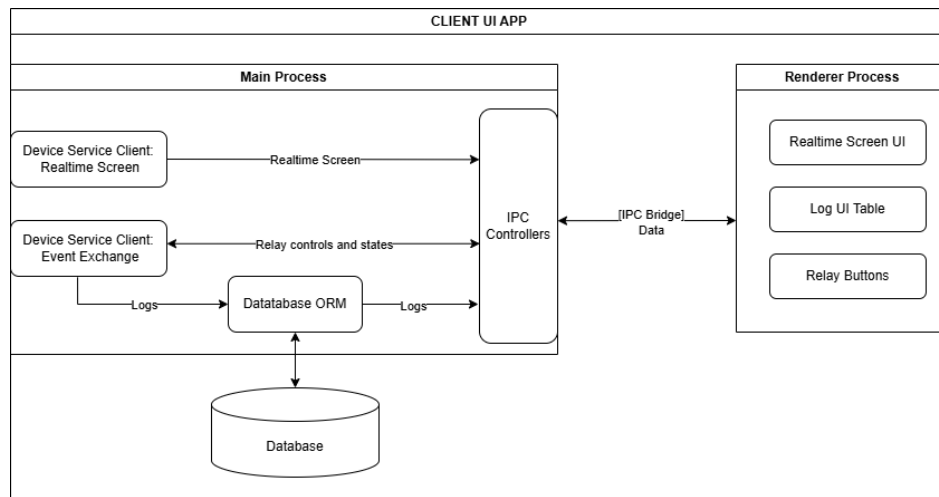
Hình 3.15. Trình tự giao tiếp phần mềm trong trường hợp thực hiện OTA

3.6 Giao diện điều khiển cho người dùng

Phần mềm giao diện được thiết kế cho người dùng cuối để lưu trữ, hiển thị và điều khiển phần mềm giải mã Device Service.

Yêu cầu thiết kế:

- Giao diện trực quan, gần gũi với người dùng.
- Độc lập với Device Service, kiến trúc có thể dễ dàng mở rộng, hướng tới trở thành công cụ trực quan có thể quan sát và hỗ trợ debug cho Device Service.
- Tương tác với cơ sở dữ liệu để đọc ghi các phiên bản.

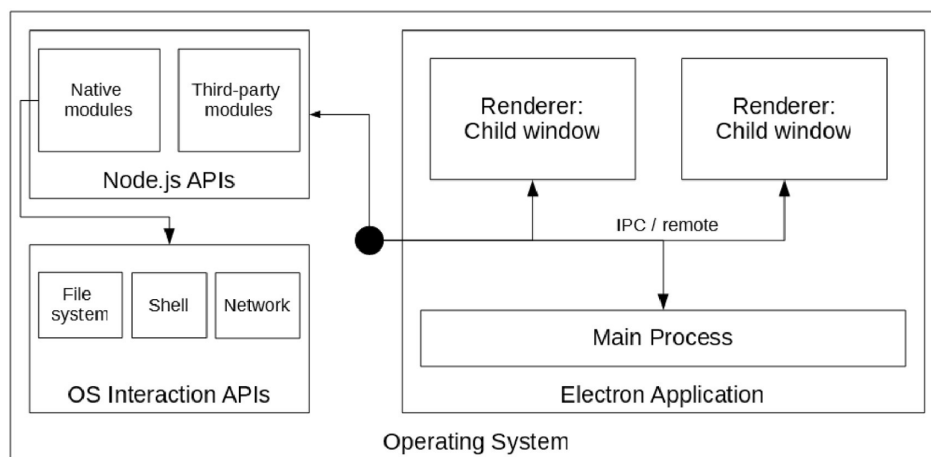


Hình 3.16. Sơ đồ khối triển khai phần mềm giao diện

Phần mềm giao diện xây dựng trên môi trường Nodejs, sử dụng framework Electron để tạo một web-based Desktop App. Phần mềm gồm 2 tiến trình (process) chính:

- Main Process có thể tương tác trực tiếp với hệ điều hành thông qua môi trường NodeJS. Sử dụng để:
 - Mở các cổng TCP hứng dữ liệu (màn hình thời gian thực, các events) từ phần mềm giải mã Device Service.
 - Xử lý các logic, tương tác với cơ sở dữ liệu: đọc và ghi các phiên bơm (log).
 - Chuyển tiếp các dữ liệu lên phía Renderer Process để hiển thị lên giao diện
- Renderer Process là tiến trình sử dụng để hiển thị giao diện, tương tác với người dùng. Tiến trình này bản chất là 1 module có lõi là Chromium. Bản chất của việc render và hiển thị sử dụng các công nghệ web browser (HTML, CSS, JavaScript) để vẽ các element. Do đó, việc tương tác trực tiếp với hệ điều hành cũng bị hạn chế, cần có Main Process và IPC (InterProcess Communication) để người dùng thực hiện các tác vụ với hệ điều hành và cơ sở dữ liệu thông qua giao diện.

Phần mềm có một cơ chế IPC (InterProcess Communication) riêng để các process giao tiếp với nhau, giúp chia sẻ dữ liệu an toàn.



Hình 3.17. Giao tiếp giữa các process của ứng dụng thông qua IPC

3.7 Kết luận chương

Trong chương này đã trình bày các khối chức năng của các thành phần hệ thống. Từ yêu cầu về chức năng đó, đưa ra thiết kế chi tiết bao gồm:

- Sơ đồ nguyên lý mạch cứng
- Sơ đồ triển khai phần mềm
- Tương tác giữa các khối trong phần mềm.

Và diễn giải các sơ đồ.

CHƯƠNG 4. TRIỂN KHAI VÀ THỬ NGHIỆM

4.1 Quy trình triển khai

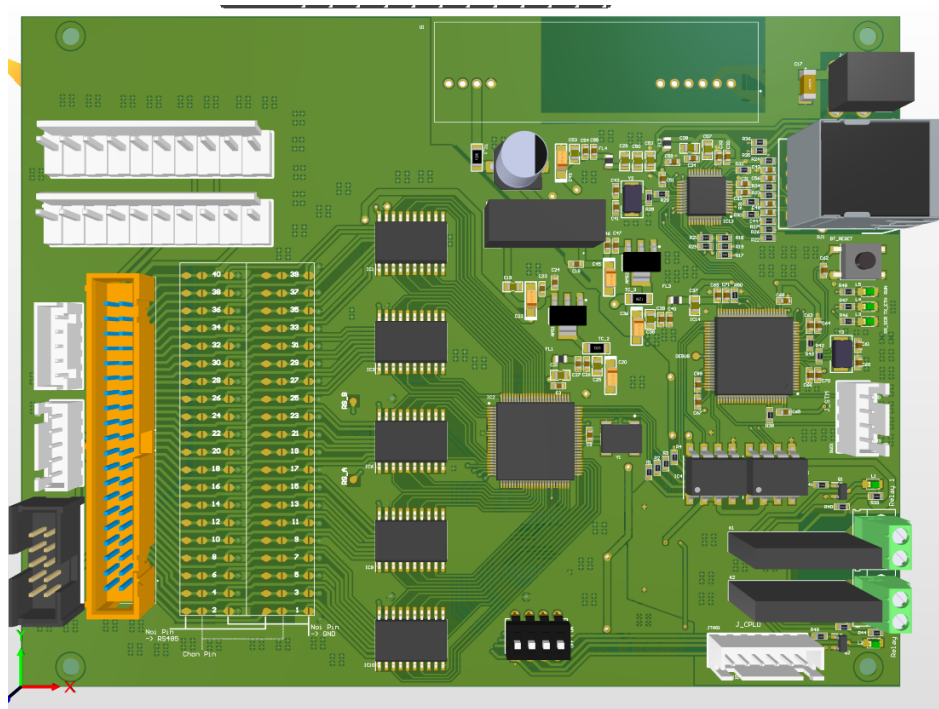
Ở chương này em trình bày chi tiết quá trình triển khai, xây dựng sản phẩm sử dụng các công nghệ đã lựa chọn ở Chương 2 và dựa trên thiết kế chi tiết đã nêu ra ở Chương 3.

Nội dung gồm quy trình triển khai, mô tả đầu ra sản phẩm, kiểm thử và chạy với dữ liệu mô phỏng, lắp ráp và kiểm thử tại hiện trường.

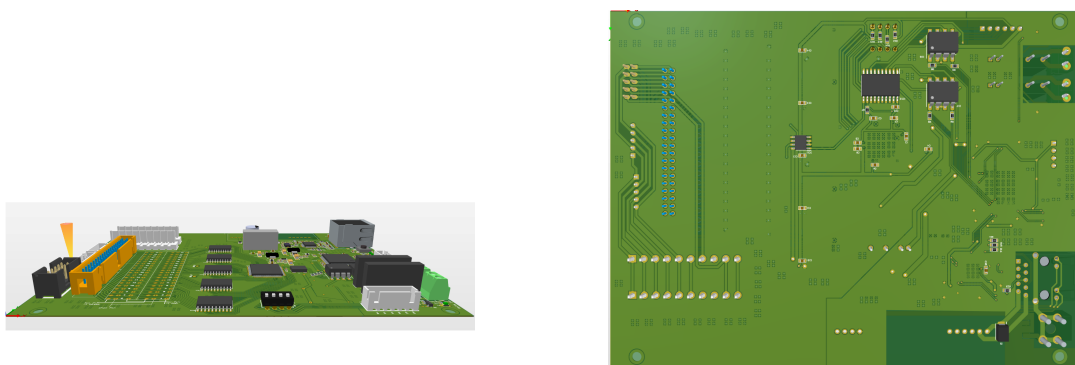
4.1.1 Mạch phân cứng thiết bị đọc màn hình

a. Vẽ mạch PCB từ sơ đồ nguyên lý

Từ sơ đồ nguyên lý đã nêu ở Chương 3, tiến hành vẽ mạch PCB trên Altium:

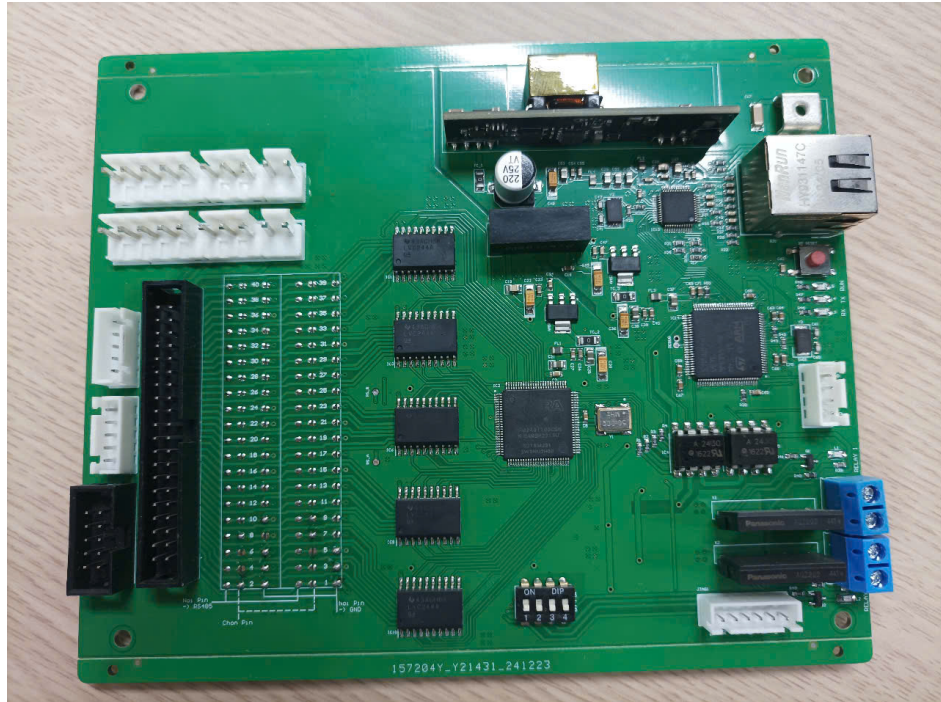


Hình 4.1. Mạch PCB - Mặt trên

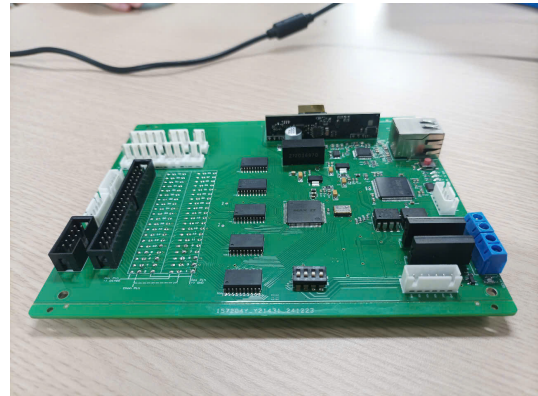
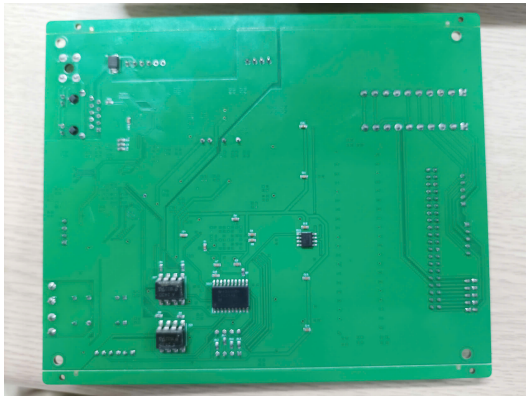


Hình 4.2. Mạch PCB - Mặt ngang và mặt dưới

b. In và hàn mạch thực tế



Hình 4.3. Sản phẩm thiết bị đọc ghi màn hình cột bơm: mặt trên



Hình 4.4. Sản phẩm thiết bị đọc ghi màn hình cột bơm: mặt ngang và dưới

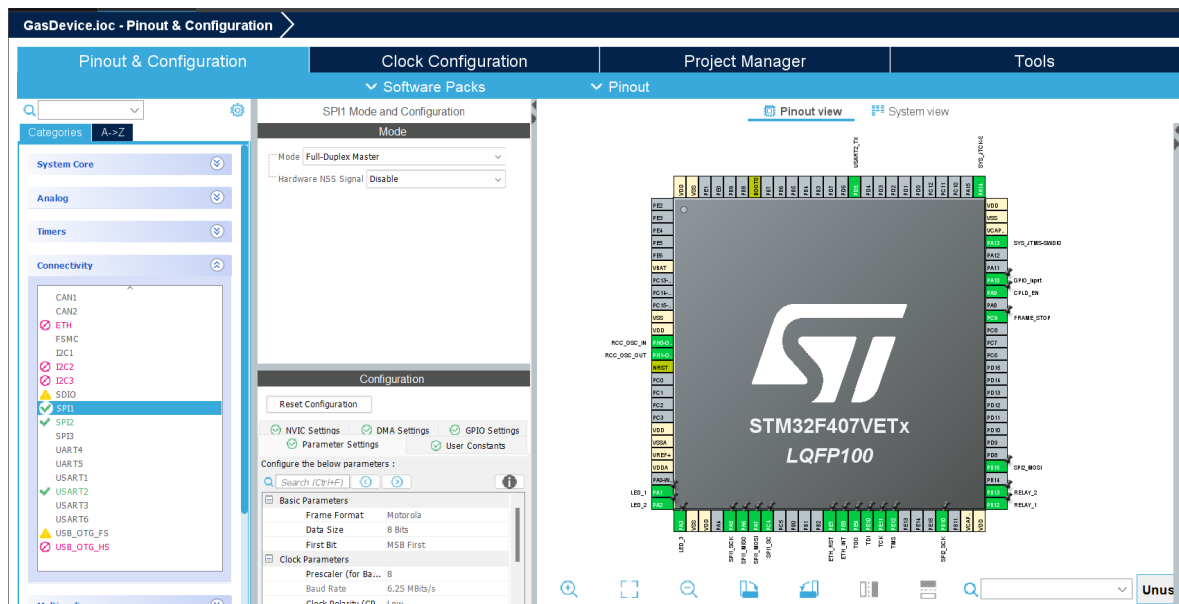
4.1.2 Firmware thiết bị đọc màn hình

Môi trường/ công cụ phát triển để lập trình cho STM32: STM32CubeIDE.

Setup các cổng IO

Các ngoại vi được sử dụng bao gồm:

- SPI1: Kết nối với module W5500, giao tiếp Ethernet.
- SPI2: Kết nối với CPLD, nhận dữ liệu màn hình dạng byte data.
- USART2: Sử dụng để debug (kết nối với máy tính và gửi thông báo lên màn hình terminal)
- Các cổng IO khác:



Hình 4.5. Triển khai: cấu hình các chân trong STM32CubeIDE

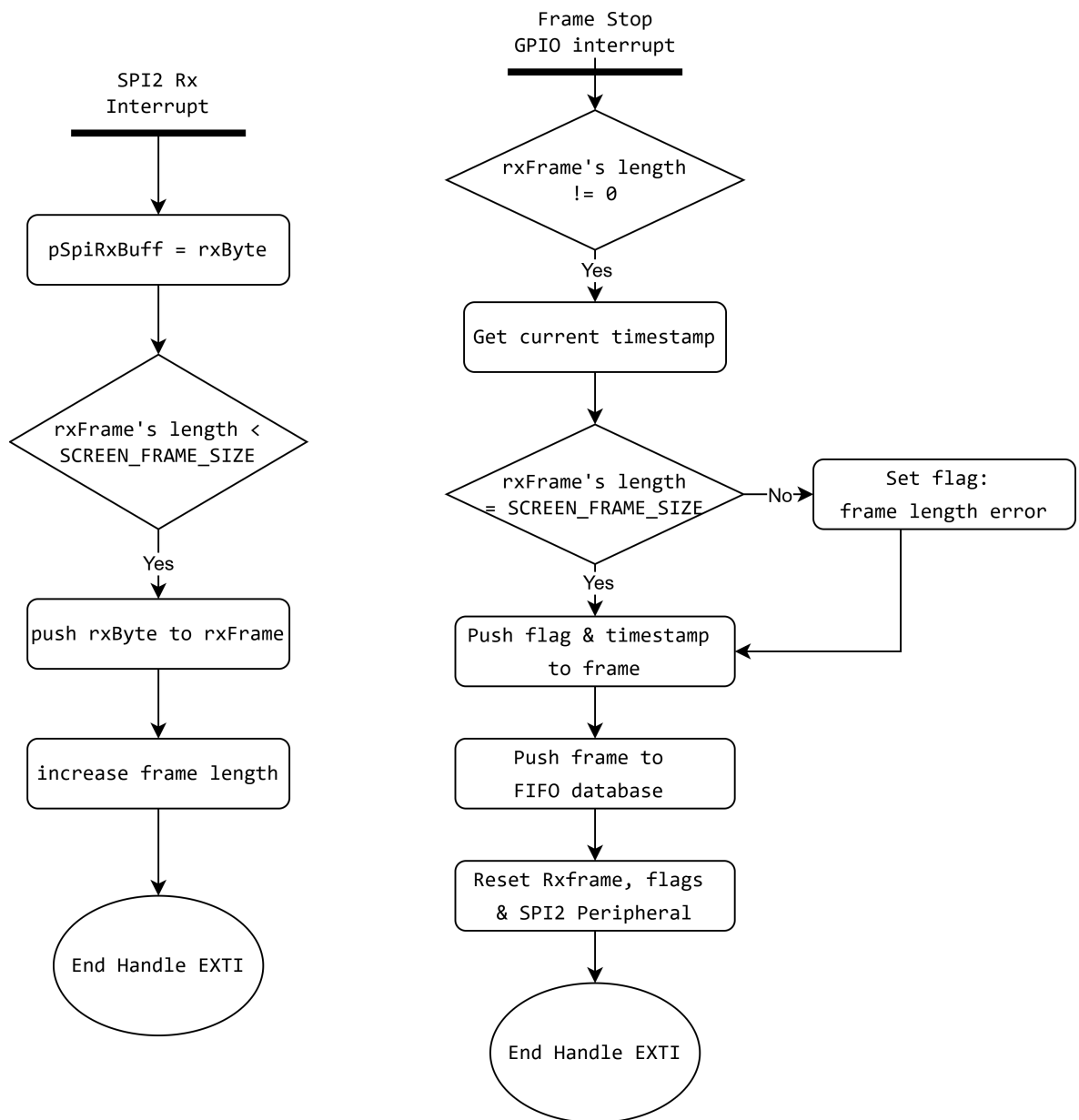
- PB12 và PB13: điều khiển relay 1 và 2.
- PA1, PA2 và PA3: các đèn báo led.
- PA9: CPLD Enable
- PC9: Frame Stop - Ngắt GPIO đầu vào báo kết thúc 1 frame

Triển khai firmware

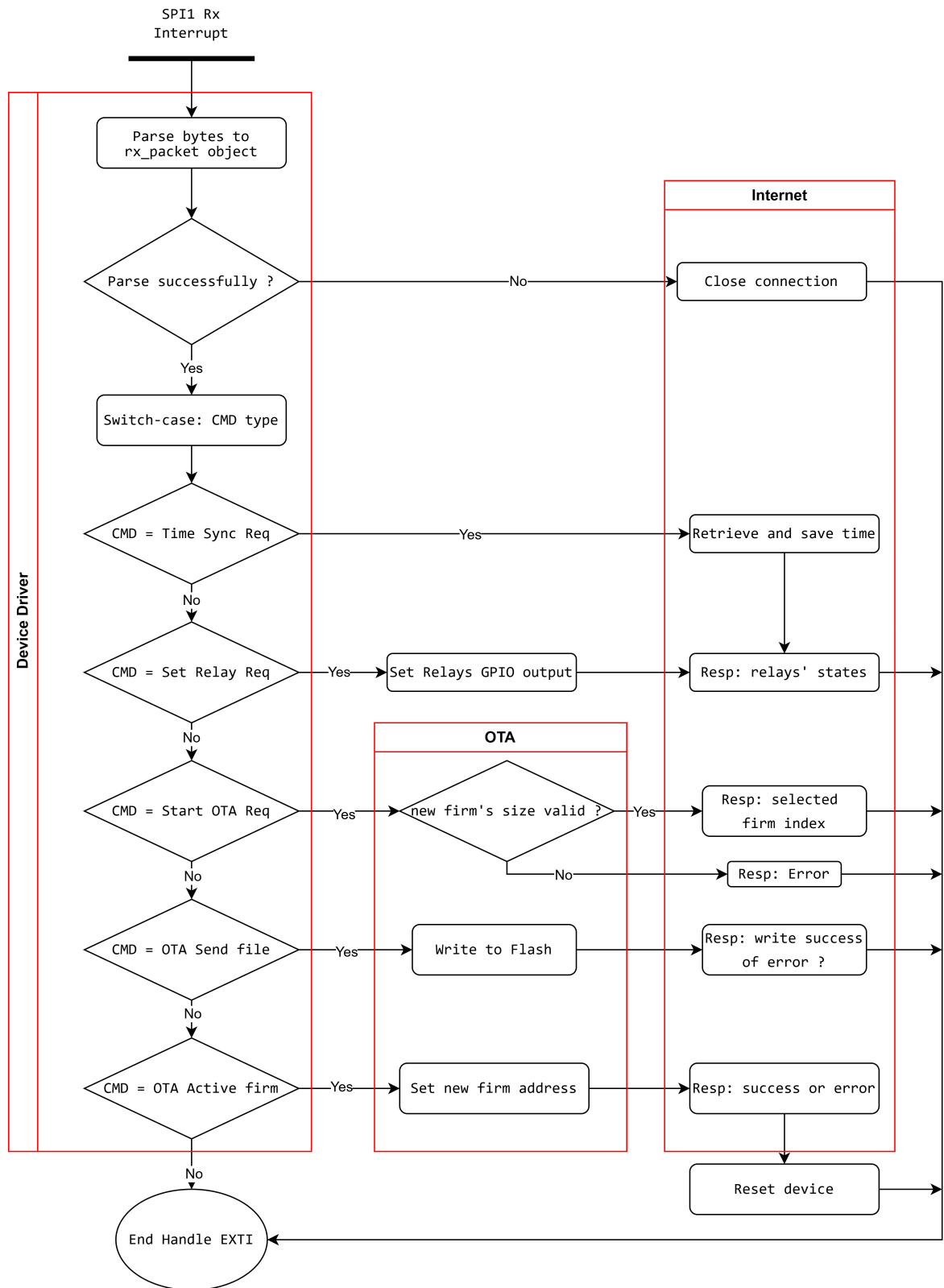
Về tổ chức các thư mục, tại thư mục core/Src tổ chức thành 4 thư mục lớn chứa 4 modules được nêu trong phần Kiến trúc triển khai firmware (xem hình minh họa ở chương trước) là: capture_driver, device_driver, internet và fota

Các thuật toán quan trọng bao gồm:

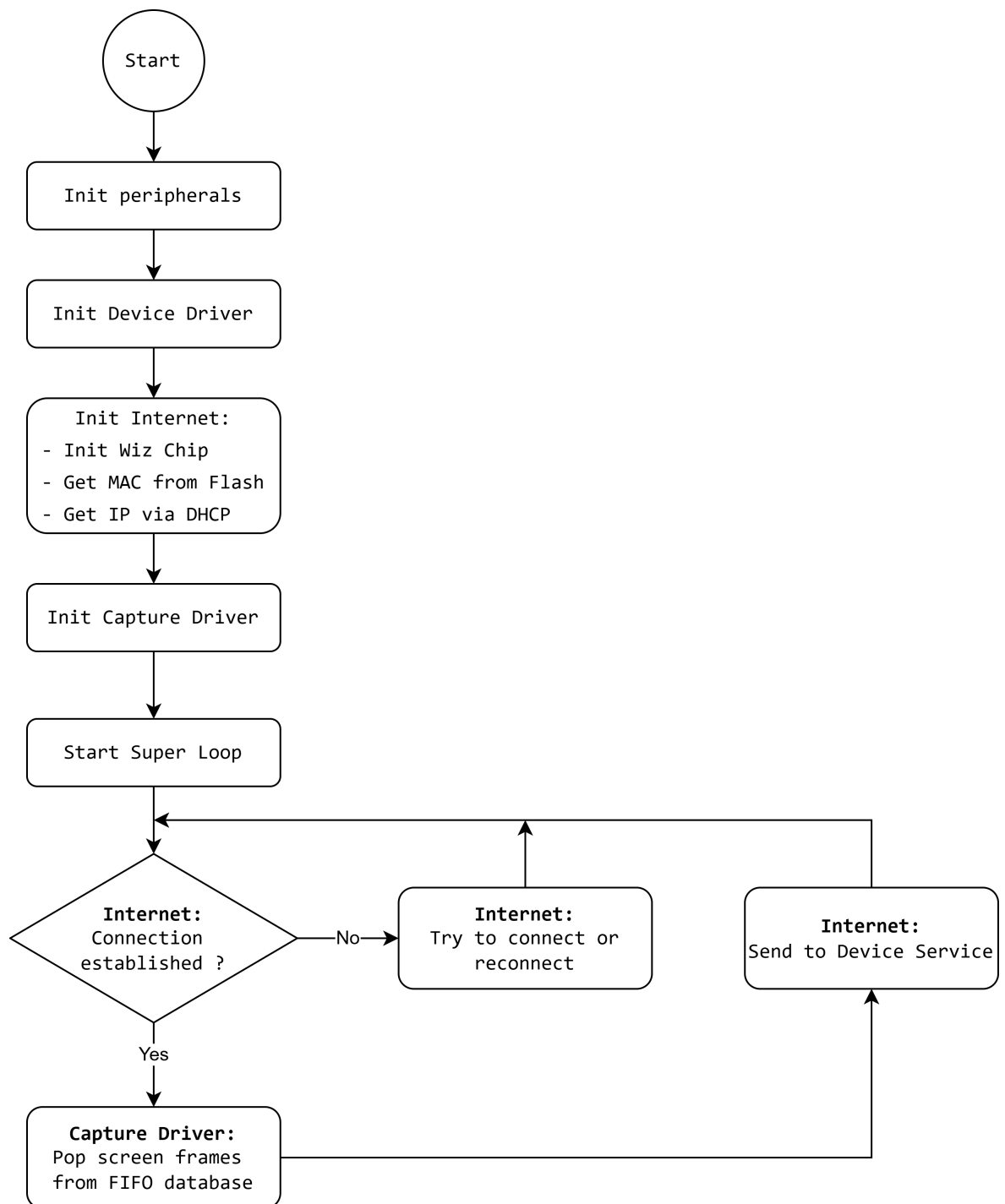
- Khởi tạo và kết nối với phần mềm Device Service
- Xử lý ngắt SPI1: Nhận yêu cầu từ Device Service
- Xử lý ngắt SPI2: Nhận SPI byte từ CPLD
- Xử lý ngắt báo Frame Stop: Báo kết thúc 1 screen frame và tiến hành lưu frame



Hình 4.6. Thuật toán xử lý ngắt nhận dữ liệu màn hình



Hình 4.7. Thuật toán xử lý ngắt nhận dữ liệu yêu cầu Device Service



Hình 4.8. Thuật toán cho chương trình chính

Cấu trúc gói tin giao tiếp giữa Thiết bị đọc ghi màn hình (Device) và Phần mềm giải mã (Device Service) được nêu trong bảng 4.1.

Trường	Kích thước	Offset	Mô tả
Packet Length	2 bytes	0–1	Tổng chiều dài gói tin trừ đi 12 byte đầu (độ dài Payload).
Device MAC	6 bytes	2–7	Địa chỉ MAC của thiết bị đọc ghi màn hình
CMD	1 byte	8	Loại lệnh (command) được gửi (xem bảng 4.2.).
Reserved	3 bytes	9–11	3 byte dự phòng, chưa sử dụng.
Payload	Tùy loại CMD	12 trở đi	Dữ liệu phụ thuộc vào từng loại CMD, ví dụ frame màn hình, trạng thái relay, v.v.

Bảng 4.1. Cấu trúc gói tin giữa Device và Device Service

Các loại Command (CMD) để giao tiếp với Thiết bị:

CMD	Tên	Mô tả
0x01	CMD_HANDSHAKE	CMD Handshake, dùng để gửi từ ‘Device’ đến ‘Device Service’.
0x02	CMD_REPORT_RAWDATA	CMD gửi frame màn hình, dùng để stream screen frame từ ‘Device’ đến ‘Device Service’.
0x03	CMD_REPORT_RELAY	CMD gửi trạng thái relay từ ‘Device’ đến ‘Device Service’.
0x04	CMD_SET_RELAY	CMD yêu cầu đặt trạng thái relay từ ‘Device Service’ đến ‘Device’.
0x05	CMD_TIME_SYNC	CMD yêu cầu đồng bộ thời gian từ ‘Device Service’ đến ‘Device’.
0x06	CMD_PING	CMD ping kiểm tra kết nối từ ‘Device Service’ đến ‘Device’.
0x07	CMD_OTA_START	CMD yêu cầu bắt đầu OTA từ ‘Device Service’ đến ‘Device’.

0x08	CMD_OTA_SEND_FILE	CMD yêu cầu gửi file OTA từ 'Device Service' đến 'Device'.
0x09	CMD_OTA_ACTIVE_FIRM	CMD yêu cầu kích hoạt firmware mới từ 'Device Service' đến Device'.
0x0A	CMD_OTA_OTA_RESP	CMD phản hồi OTA từ 'Device' đến 'Device Service'.

Bảng 4.2. Danh sách các loại CMD trong enum Cmd_t

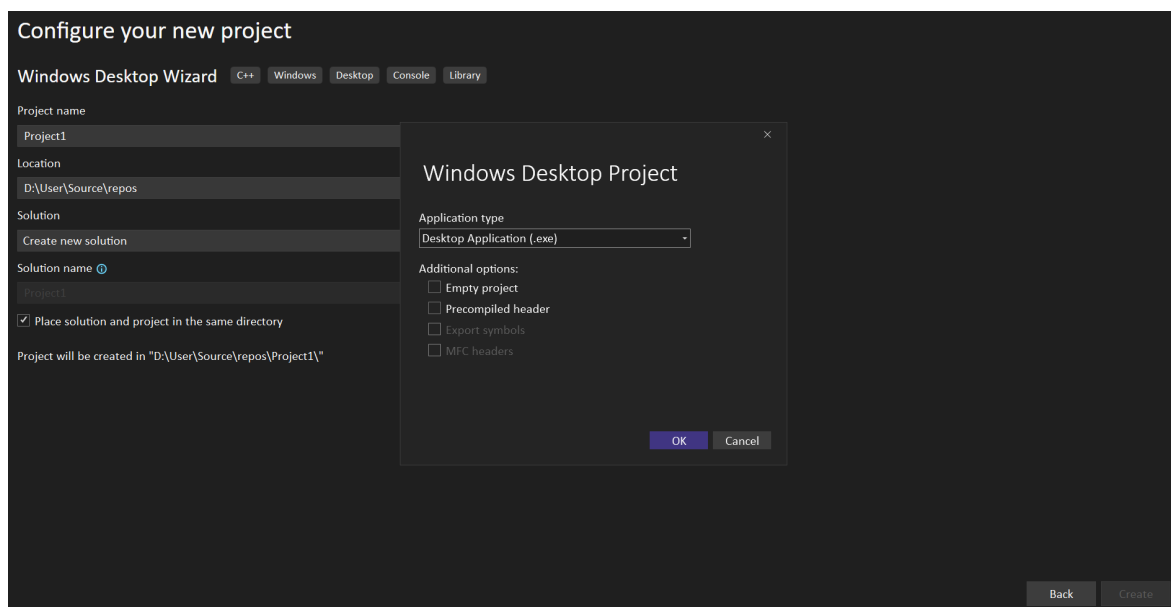
4.1.3 Phần mềm giải mã Device Service

Chuẩn bị môi trường:

- Môi trường phát triển: Visual Studio.
- Ngôn ngữ: C++.
- Loại dự án: Win32 API C++.
- Các thư viện được sử dụng: Boost,

Các bước triển khai:

Bước 1: Tạo 1 Win32 Application Project



Hình 4.9. :Tạo C++ Win32 Project trên Visual Studio

Bước 2: Triển khai chế độ chạy cho phần mềm.

Triển khai để khi phần mềm được khởi chạy sẽ:

- Tạo 1 mutex riêng có tên cố định trên Window, đảm bảo dù nhấn mở phần mềm nhiều lần thì vẫn sẽ chỉ có 1 Instance của App đang chạy.
- Phần mềm không có giao diện, sẽ là 1 tác vụ chạy ngầm trên Window (Window Service).

Bước 3: Viết các khối triển khai.

Các khối triển khai tuân theo thiết kế ở hình [3.12](#).

Phần này nêu chi tiết những thuật toán quan trọng cho các khối

Thuật toán bộ giải mã và phát hiện trạng thái máy

Khối thực hiện: Logger.

Một ‘log’ (lượt bơm) được chốt sau khi giá trị màn hình (số lít và giá tiền) dừng tăng trong 5 giây, hoặc cột đang bơm thì dừng và reset màn hình.

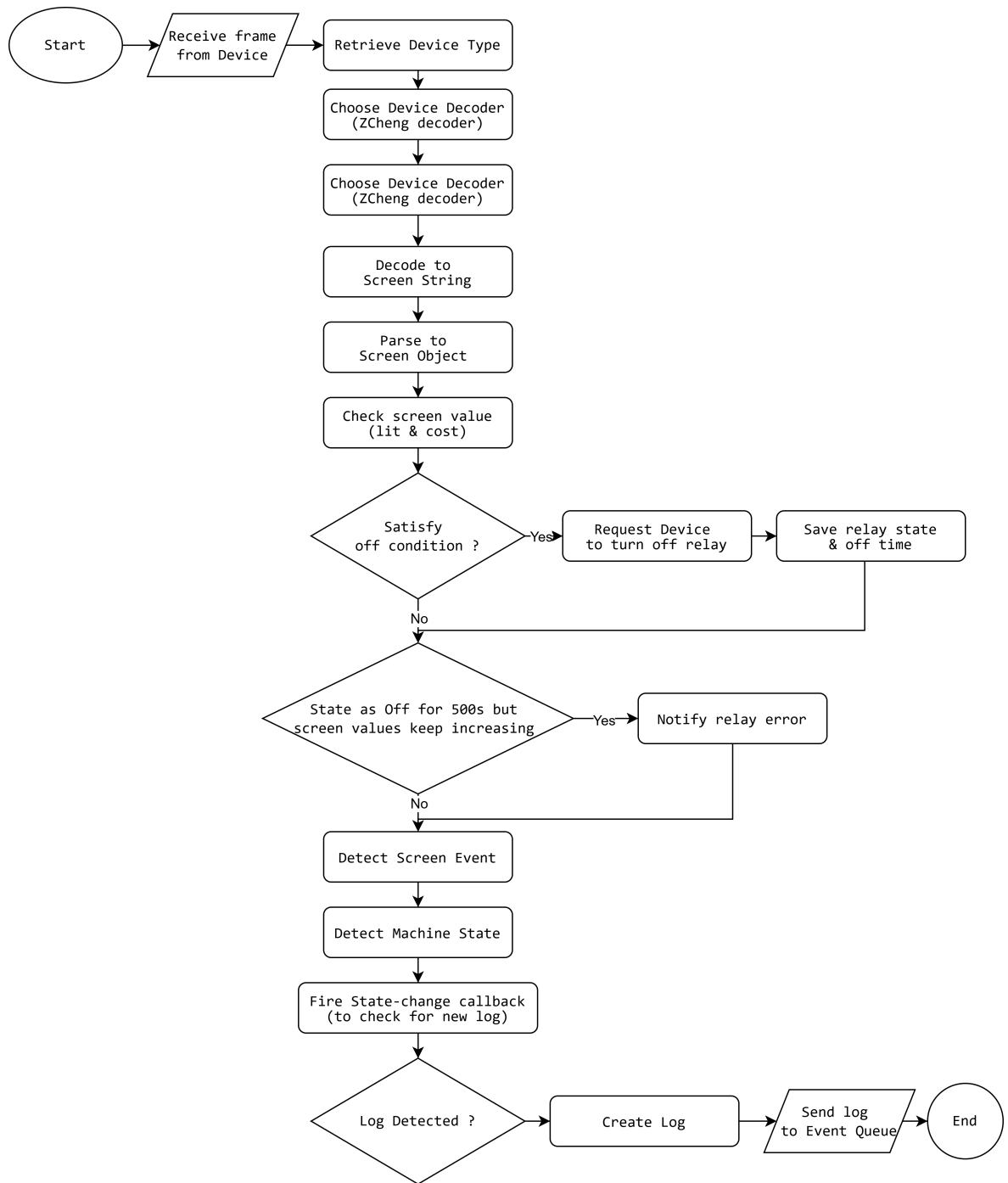
Một ‘log’ cũng có thể dùng để lưu màn hình hiển thị giá trị preset (đặt trước lượng xăng cần bơm)

Một ‘sub log’ cũng là một loại ‘log’, để bổ sung thông tin cho một ‘log’ đã có trong trường hợp việc bơm tạm dừng quá 5 giây nhưng lại được bơm tiếp thay vì reset.

Một ‘log’ đi cùng với (các) ‘sublog’ của nó sẽ tạo thành 1 hóa đơn hoàn chỉnh.

Trong trường hợp màn hình lỗi xảy ra, dữ liệu cũng sẽ được lưu vào 1 sublog.

Thuật toán giải mã màn hình phát hiện lượt bơm (Log Process):



Hình 4.10. Thuật toán giải mã màn hình và phát hiện lượt bơm

Các Screen Event (bảng 4.4.) được xác định sau khi giải mã 1 frame màn hình, dùng để phân loại sự kiện màn hình

LogInfo	
uint8_t	logInfoType
uuid	logId
uuid	subLogId
double	cost
double	volume
double	price
double	preCost
double	preVolume
double	prePrice
uint64_t	timestamp
string	subLogContent

Hình 4.11. Các trường dữ liệu trong một Log hoặc Sublog

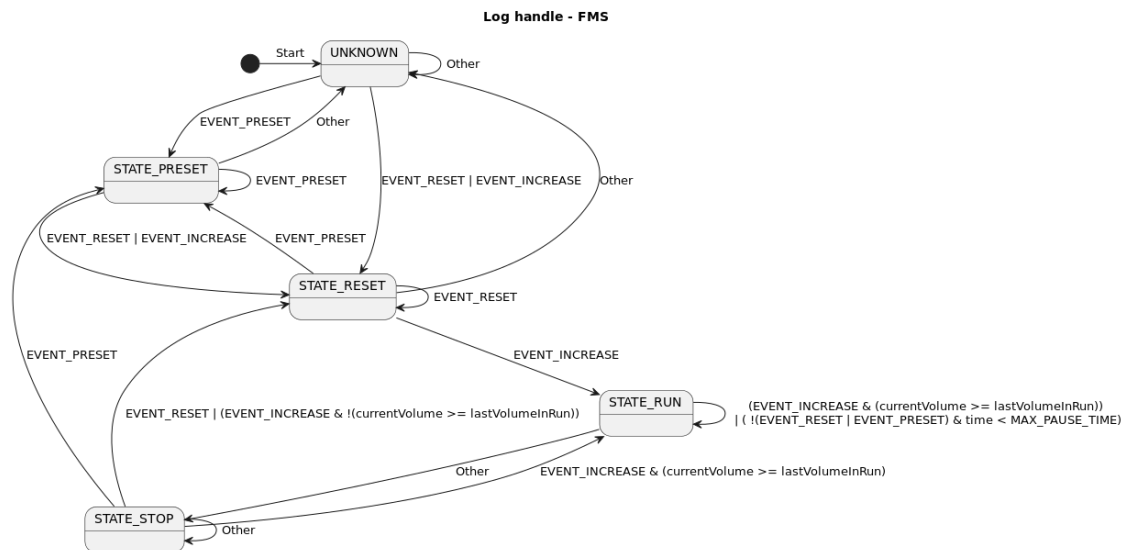
Event	Mô tả
EVENT_NONE	Giá trị khởi tạo khi chưa thực hiện xác định Event
EVENT_UNKNOWN	Event không xác định được
EVENT_RESET	Báo reset màn hình
EVENT_PRESET	Báo preset (đặt trước giá trị màn hình)
EVENT_INCREASE	Giá trị màn hình (lít và tổng tiền) đang tăng
EVENT_PAUSE	Giá trị màn hình (lít và tổng tiền) đang dừng
EVENT_ERROR	Màn hình lỗi

Các loại trạng thái máy (Machine State, bảng 4.4.), được phát hiện dựa theo trạng thái máy trước đó kết hợp với sự kiện màn hình mới nhất.

Trạng thái	Mô tả
STATE_UNKNOWN	Không rõ, chưa xác định được trạng thái máy
STATE_RESET	Báo màn hình đã được reset (giá và thể tích đặt về 0)
STATE_PRESET	Báo người dùng đã preset (đặt trước) số lít hoặc số tiền
STATE_RUN	Đang trong quá trình bơm, giá trị trên màn hình đang tăng
STATE_STOP	Quá trình bơm đang tạm dừng, giá trị trên màn hình không tăng
STATE_ERROR	Trạng thái lỗi, phát hiện màn hình lỗi

Bảng 4.4. Mô tả các trạng thái máy (Machine State)

Việc phát hiện sự kiện màn hình và chuyển đổi trạng thái máy được chỉ ra trong sơ đồ trạng thái (hình 4.12.). Từ đó theo dõi được tình trạng cột bơm.



Hình 4.12. Sơ đồ trạng thái cho cột bơm

Sau khi xác định trạng thái mới, hàm callback tương ứng được gọi để kiểm tra phiên bơm mới nếu có:

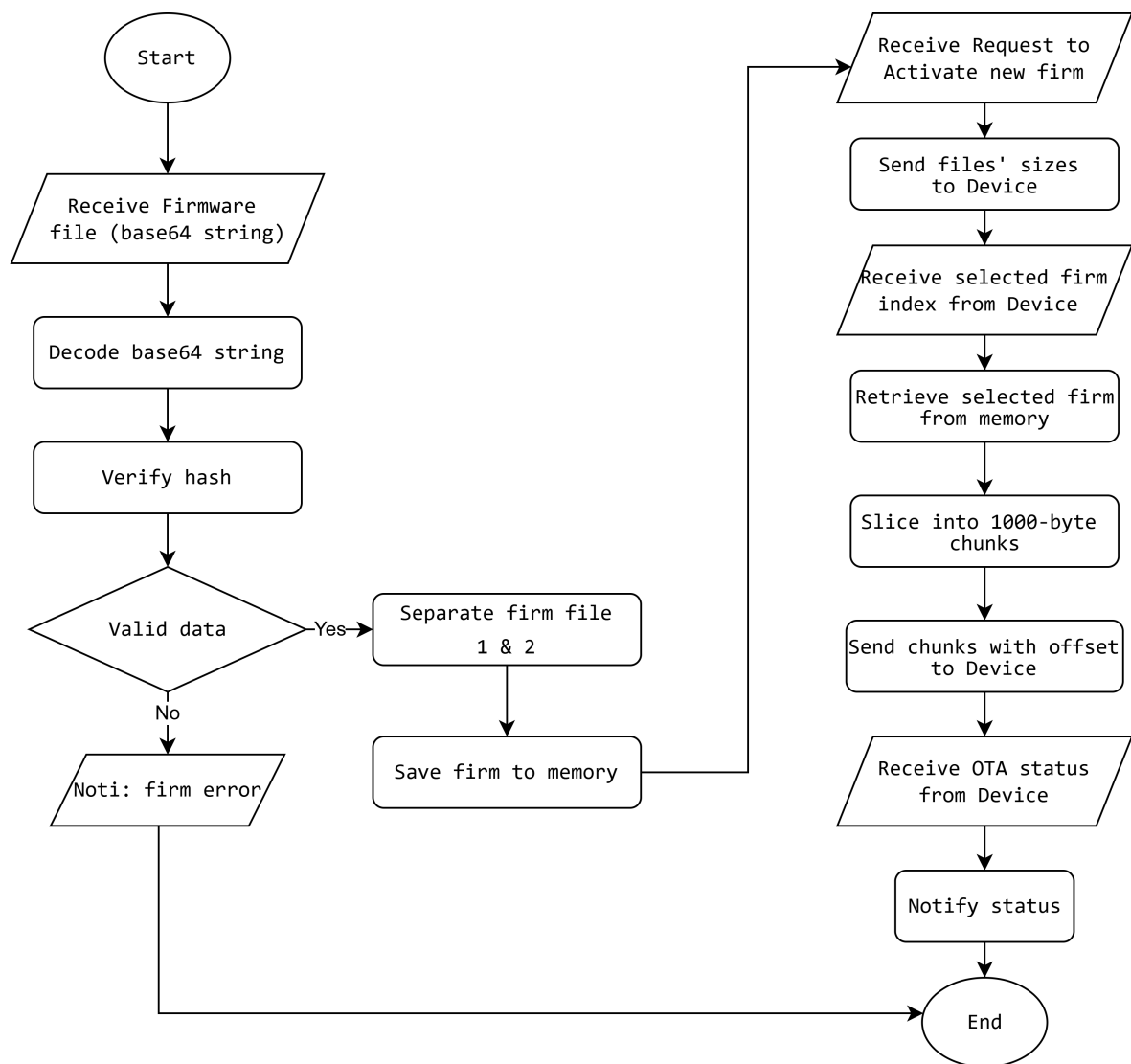
Trạng thái cũ	Trạng thái mới	Hàm callback	Mô tả
STATE_PRESET	STATE_PRESET	Check overtime to cut preset	Kiểm tra xem màn hình ở trạng thái PRESET đã được giữ hơn 5 giây chưa. Nếu có, tạo một log mới cho preset.
STATE_RESET	STATE_RUN	Generate new Log	Tạo mới một log trống để bắt đầu ghi nhận dữ liệu màn hình.
STATE_RUN	STATE_RUN	Update Log info	Trong khi vẫn ở trạng thái RUN, hệ thống cập nhật thông tin log (ví dụ: thời gian, tiền, lít, đơn giá...).
STATE_RUN	STATE_STOP	Cut Log	Khi chuyển từ RUN sang STOP, hệ thống chốt log hiện tại

Bảng 4.5. Callback được gọi sau khi xác định trạng thái mới

Các firmware file được gửi tới Device Service dạng base64-string, được kiểm tra tính hợp lệ sau đó lưu vào bộ nhớ tạm.

Khi có yêu cầu kích hoạt firmware, Device Service tiến hành:

- Hỏi Thiết bị đọc màn hình file firmware cần cập nhật
- Chia file firmware được chọn dưới dạng các chunk độ dài 1000-byte
- Gửi từng chunk tới Thiết bị đọc ghi màn hình cùng offset.



Hình 4.13. Thuật toán thực hiện OTA trên Device Service

Thuật toán thực hiện OTA

API cho Client để giao tiếp với Device Service

Device Service cung cấp API mở để bên Client UI hoặc các bên Client khác có thể giao tiếp để:

- Nhận dữ liệu thời gian thực và các phiên bơm (log), trạng thái relay
- Yêu cầu cập nhật relay hoặc thực hiện OTA.

Dạng dữ liệu: JSON.

Giao thức: TCP.

Mỗi request message là 1 JSON bao gồm các trường

- Message ID: UUID định danh message
- Message Type: Loại message để phân loại nội dung.
- Và các trường khác là payload tùy thuộc vào Message Type.

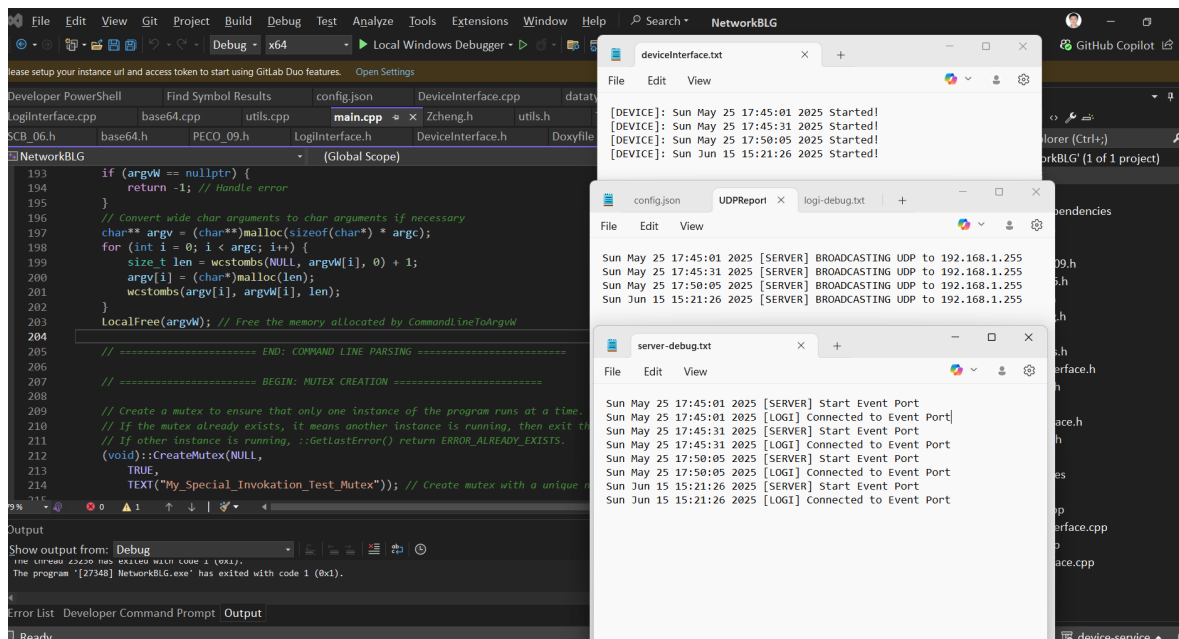
Các Message Type được hỗ trợ:

MSG Type	Mô tả
0	Báo cáo sự kiện Log/SubLog từ Device Service tới UI Client. Bao gồm thông tin log hoặc sublog được chốt trong quá trình sử dụng thiết bị.
1	Báo cáo sự kiện đặt Preset từ Device Service tới UI Client. Được gửi khi nội dung preset không đổi trong 5s hoặc khi chuyển từ preset về reset.
2	Yêu cầu từ UI Client tới Device Service để thay đổi trạng thái bật/tắt Relay và điều kiện tự động tắt Relay.
3	Device Service báo cáo trạng thái của các Relay về UI Client. Có thể là phản hồi cho yêu cầu Set Relay hoặc báo cáo chủ động khi có thay đổi.
5	UI Client gửi yêu cầu tải xuống file firmware về Device Service. File dưới dạng base64, chứa nội dung cần cập nhật cho thiết bị.
6	UI Client yêu cầu Device Service kích hoạt firmware mới đã được tải xuống. Sau khi thực hiện, thiết bị sẽ tự động reset.
7	UI Client yêu cầu Device Service trả về danh sách trạng thái hiện tại của tất cả các thiết bị đang quản lý.
8	Device Service chủ động báo cáo trạng thái của một thiết bị đọc ghi màn hình (kết nối, loại thiết bị, phiên bản firmware...) về cho UI Client.

Bảng 4.6. Danh sách các message_type của Device Service

Tạo các file log để quan sát tiến trình chạy (hình 4.14.):

- `deviceInterface.txt`: file log cho khối Device Interface, ghi các thiết bị đã kết nối, kèm thông báo và lỗi trong quá trình chạy.
- `server-debug.txt`: file log thông báo và lỗi trong quá trình kết nối và giao tiếp Client UI.
- `UDPReport.txt`: Thông báo hiện trạng của tiến trình broadcast UDP.



Hình 4.14. Device Service: Các file logs

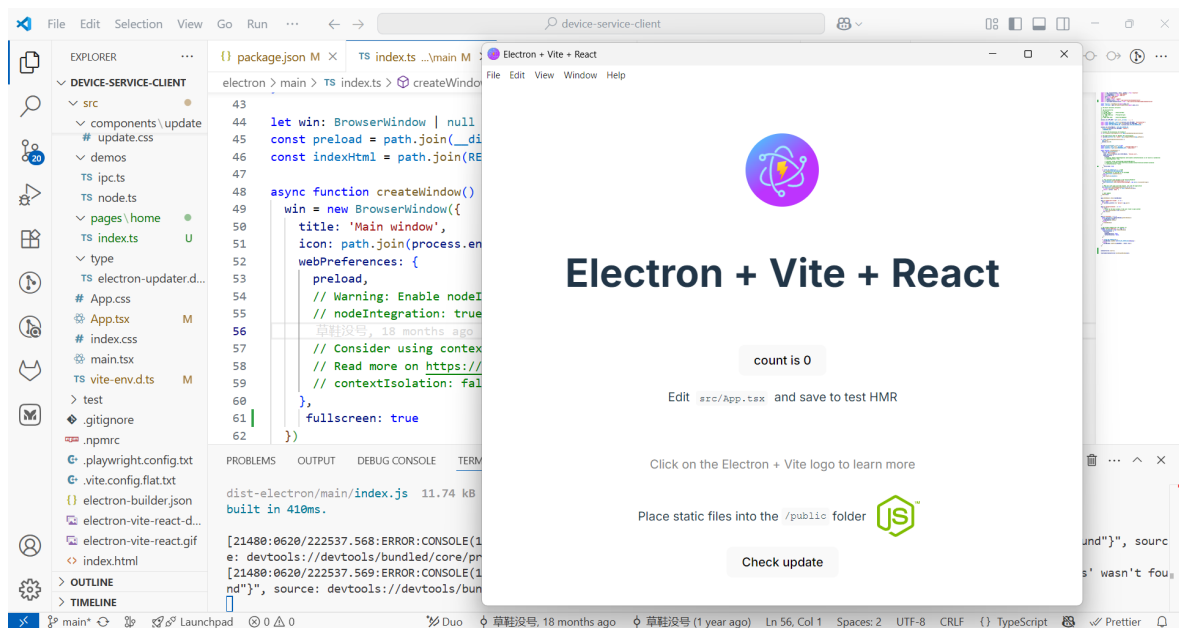
4.1.4 Viết phần mềm Client UI hiển thị các phiên bom

- Môi trường sử dụng: NodeJS v20.18.1
- Ngôn ngữ: TypeScript
- Database: SQLite
- Các thư viện chính sử dụng: ReactJS, sqlite3, MUI Components

Các bước triển khai:

Bước 1: Tạo cấu trúc dự án Electron

- Sử dụng câu lệnh `npm create electron-vite@latest` để setup dự án.
- Chọn template là ReactJS, ngôn ngữ TypeScript, nhấn enter để bắt đầu tạo mẫu.



Hình 4.15. Mẫu dự án sử dụng Electron + Vite + ReactJS

Bước 2: Triển khai các tác vụ ở Main Process bao gồm:

- Tạo 2 TCP Client:
 - Realtime Screen Device Service Client: để nghe dữ liệu màn hình thời gian thực, chuyển tiếp màn hình tới Render Process ở cổng 5001
 - Event Device Service Client: để nhận phiên bơm (log), điều khiển relay ở cổng 5002. Các phiên bơm nhận được sẽ được chuyển tiếp tới Database ORM.
- Tạo Database ORM để đọc ghi cơ sở dữ liệu phiên bơm.

Cơ sở dữ liệu phiên bơm gồm 1 bảng lưu các log/sublog gồm các trường:

LogInfo	
uint8_t	logInfoType
uuid	logId
uuid	subLogId
double	cost
double	volume
double	price
double	preCost
double	preVolume
double	prePrice
uint64_t	timestamp
string	subLogContent

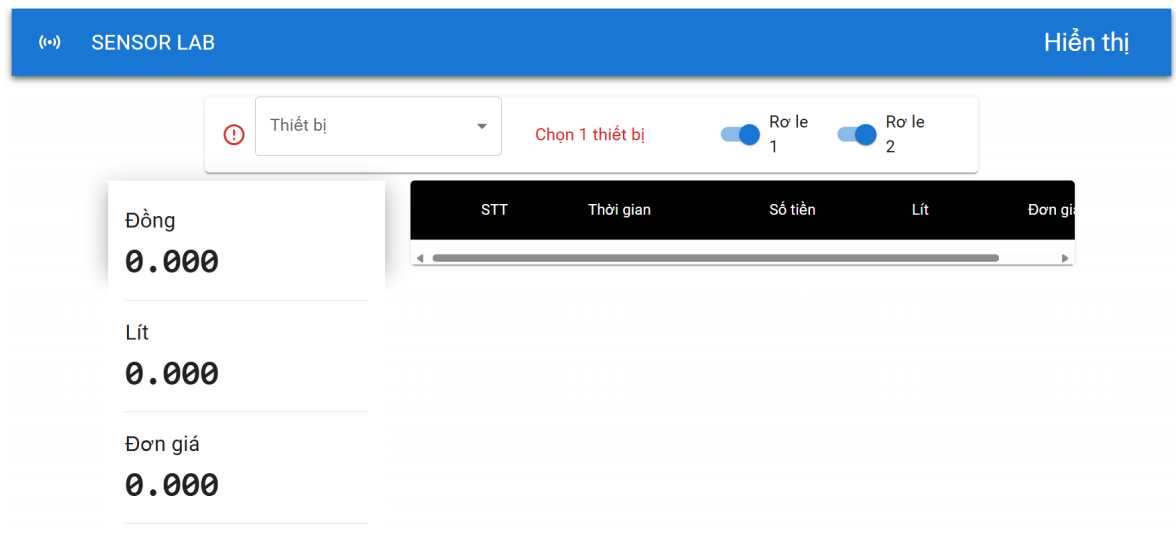
Hình 4.16. Các trường lưu trong bảng Log của Database

Bước 3: Triển khai giao diện tại Renderer Process

Sử dụng MUI để xây dựng các component:

- Khung hiển thị màn hình thiết bị (3 dòng).
- Bảng thông kê log/sublog theo thời gian thực.
- Lọc, tìm kiếm theo device_id, log_id, thời gian, trạng thái, ...
- Hiển thị trạng thái kết nối và relay theo từng thiết bị. Các nút điều khiển relay

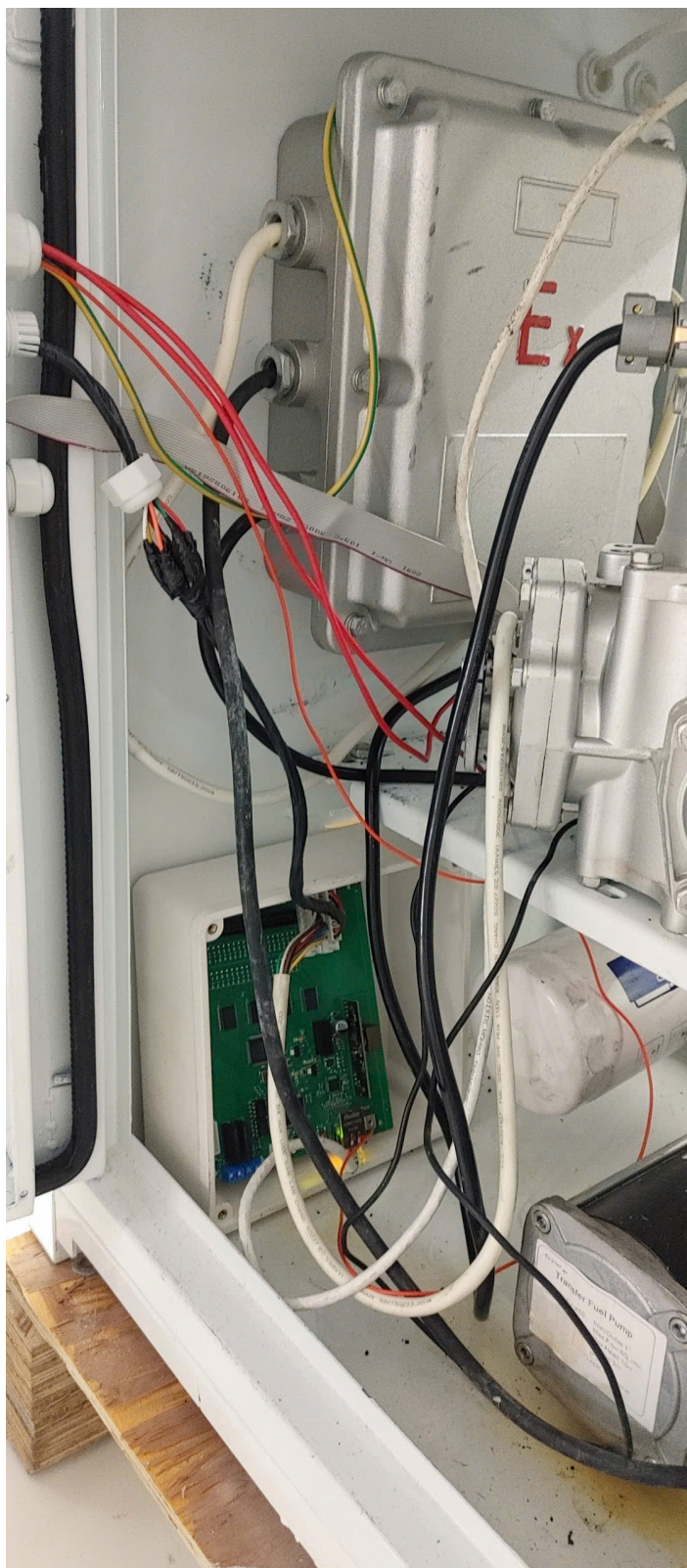
Giao diện hoàn chỉnh



Hình 4.17. Giao diện hiển thị Client UI

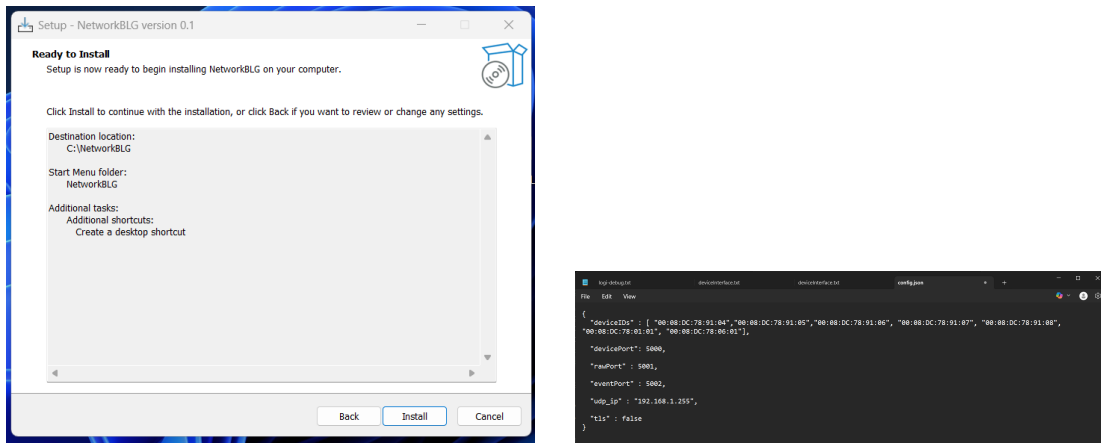
4.2 Thử nghiệm và đánh giá kết quả thực tế

Tiến hành lắp mạch vào cột bơm (hình 4.18.):



Hình 4.18. Lắp thiết bị đọc màn hình vào cột bơm

Cài đặt Device Service tại trạm xăng, cấu hình danh sách các thiết bị trong trạm (hình [4.19.](#))



Hình 4.19. Cài đặt và cấu hình cho Device Service tại hiện trường

Các tiêu chí đánh giá:

Tiêu chí đánh giá	Phương pháp đánh giá	Kết quả
Device Service broadcast UDP và các thiết bị kết nối được tới Device Service	Kiểm tra File log deviceInterface.txt liệt kê các thiết bị đã kết nối	Đạt
Hiển thị dữ liệu màn hình theo thời gian thực	Tiến hành bơm và so sánh màn hình cây xăng thực và màn hình Client UI, kiểm tra hiển thị tương đồng	Đạt
Lưu LogSubLog vào cơ sở dữ liệu	Giá trị Logsublog giống với phiên đã bơm, được lưu trong cơ sở dữ liệu	Đạt
Cập nhật trạng thái Relay	Thử nhấn nút bật tắt relay tại giao diện, kiểm tra đèn báo relay và thử tiếp tục bơm khi relay tắt, thấy trạng thái relay trên giao diện và thực tế đồng bộ	Đạt
Hoạt động ổn định	Không gặp lỗi treo ứng dụng trong quá trình thử nghiệm nhiều thiết bị. Cho chạy liên tục trong nhiều tuần	Đạt

Bảng 4.7. Bảng kết quả đánh giá kiểm thử hệ thống

4.3 Kết luận chương

Chương này đã chỉ ra quy trình triển khai cụ thể cho các thành phần hệ thống, đồng thời trình bày quá trình đánh giá kết quả thực tế khi lắp đặt hệ thống tại hiện trường. Từ đó, liệt kê cụ thể các chức năng hệ thống đã đạt được.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Nhìn chung, đồ án đã hoàn thành một hệ thống bao gồm mạch cứng và phần mềm giải mã cho Thiết bị đọc ghi màn hình cột bơm xăng dầu với những chức năng, yêu cầu cơ bản, được đánh giá, kiểm tra tại hiện trường.

5.1 Tóm tắt kết quả đã đạt được

Nhìn chung đồ án đã đưa ra thiết kế và trình tự triển khai cho Thiết bị đọc ghi màn hình cột bơm xăng dầu đáp ứng đủ chức năng cốt lõi là giải mã màn hình và phát hiện phiên bơm cho cụ thể loại cột bơm ZCheng. Thiết bị cũng đã được kiểm tra tại hiện trường, đảm bảo hoạt động được. Ngoài ra, các chức năng/tiện ích khác như điều khiển đóng ngắt relay, OTA cũng đã được thiết kế và triển khai.

Đồ án cũng đưa ra phần cứng có khối đầu vào và bộ chuyển đổi đáp ứng được các loại đầu vào màn hình khác nhau, tạo tiền đề để mở rộng hệ thống, giải mã nhiều loại màn hình cây xăng cũ khác.

5.2 Những hạn chế còn tồn đọng

Giao diện cho người dùng vẫn chỉ dừng lại ở hiển thị và điều khiển cơ bản là hiển thị màn hình thời gian thực cũng như bật tắt relay. Tiến trình OTA vẫn chưa được hỗ trợ đầy đủ và tiện lợi trên giao diện. Giao diện vốn được triển khai sử dụng công nghệ Electron, một công nghệ web-based chiếm nhiều tài nguyên trên máy trong quá trình chạy.

Thiết bị mới chỉ giải mã được 1 loại cây xăng cũ phổ biến là ZCheng, trong khi trên thị trường vẫn còn một vài loại cây xăng cũ khác cần giải mã.

5.3 Định hướng phát triển trong tương lai

Ưu tiên trước tiên sẽ là tiến hành giải mã các loại cây xăng cũ khác (VD máy hãng SEEN, PECO, codetec, v.v..) hỗ trợ đa dạng các loại cây xăng.

Tiến tới phát triển các tính năng gần gũi với người dùng qua giao diện: Ví dụ các tính năng cảnh báo khi lượng xăng dầu bơm vượt ngưỡng.

Kết luận chương Chương 5 tổng kết quá trình thiết kế và phát triển thiết bị đọc ghi màn hình cột bơm xăng dầu, những kết quả ban đầu tích cực là nền tảng cơ sở cho việc phát triển hệ thống sau này, đóng góp vào quá trình chuyển đổi số cho các doanh nghiệp trong ngành xăng dầu nói chung.

TÀI LIỆU THAM KHẢO

- [1] B. Wells, “First gas pump and service station,” 5 2025. [Online]. Available: <https://aoghs.org/transportation/first-gas-pump-and-service-stations/>
- [2] L. N. Hải, L. H. Sâm, N. T. Đường, N. Q. Cường, and T. V. Tuấn, *Điện tử số*. NXB Giáo dục Việt Nam, 2008.

PHỤ LỤC